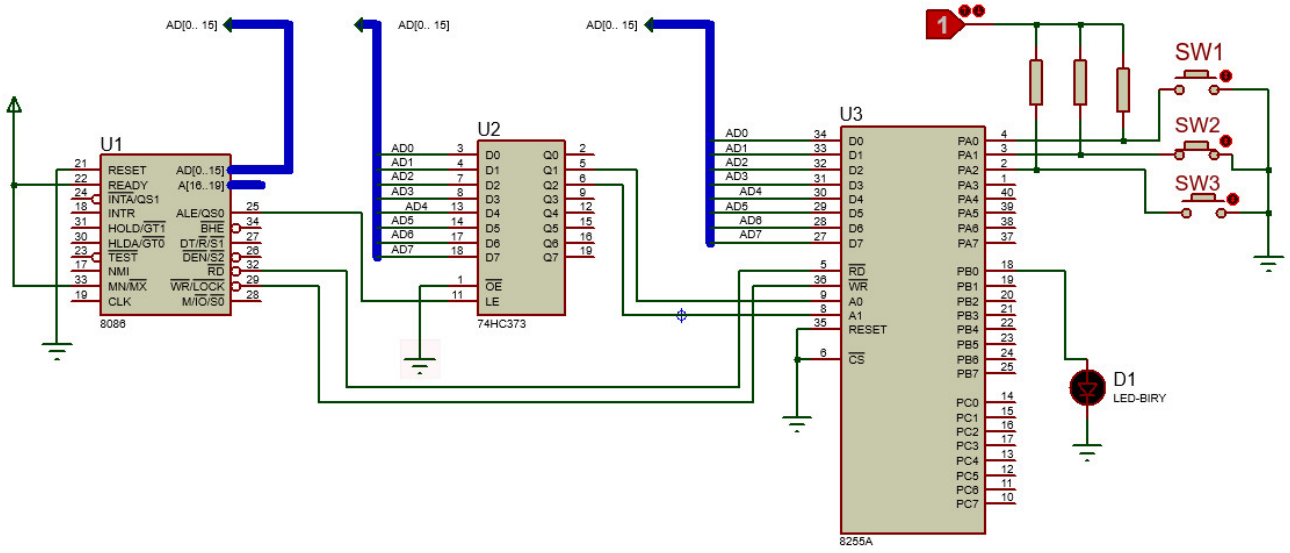
كلية العلوم - أكاديـر
ⵜⴰⵖⵡⴰⵍⵜ ⵏ ⵜⵣⴰⵔⵉⵡⵉⵏ - ⴰⴳⴰⴷⵉⵔ
FACULTÉ DES SCIENCES - AGADIR

Département de Physique

Licence PC-P
Filière Électronique & Systèmes

Semestre 6

Travaux pratiques de module : Systèmes à microprocesseur & à microcontrôleur



Responsable : Prof. Said AGOUNAD

2025-2026

Note : Les séances de travaux pratiques ont pour but essentiel la mise en application de certaines notions acquises lors des séances de cours et de travaux dirigés. Il est, sans rappeler, impératif de préparer la partie théorique de chaque manipulation avant la séance de travaux pratiques.

Quelques rappels théoriques sont délibérément détaillés dans le but d'apporter un support au cours de Systèmes à Microprocesseur et à Microcontrôleur.

Note : Science is a great weapon. From the early days until now and in the future only science can improve the life of humanity and bring enlightenment to them. Currently, there are many challenges in the world, so only young talented scientists can find effective solutions to them.

Table des matières

Partie I – Systèmes à microprocesseur **1**

Introduction : Présentation générale du µp 8086 de l'émulateur 8086 . . . . . **2**
I Présentation générale du microprocesseur Intel 8086 2
I.1 Architecture externe du µp 8086 2
I.2 Architecture interne du µp 8086 2
I.2.1 Unité d'interface de bus 4
I.2.1.1 Registres de segments 4
I.2.1.2 Pointeur d'instruction 5
I.2.1.3 Bloc logique de génération d'adresses et de contrôle des bus 5
I.2.1.4 File d'attente d'instructions 5
I.2.2 Unité d'exécution 5
I.2.2.1 Unité arithmétique et logique 5
I.2.2.2 Registres généraux 6
I.2.2.3 Registres de pointeurs et d'index 6
I.2.2.4 Registre d'état 7
II Présentation de l'émulateur 8086 : Emu8086 9

TP 1 : Prise en main de l'émulateur 8086 et programmation en assembleur . . . **13**
I Objectifs de TP 13
II Préparation théorique 13
III Manipulation 14
III.1 Modes d'adressage 14
III.2 Opérations arithmétiques et logiques 15
III.2.1 Opérations arithmétiques 15
III.2.2 Opérations logiques 16
III.3 Boucles et instructions de branchement : Opérations sur les tableaux 17
III.4 Instructions de décalage et rotation 18

TP 2 : Manipulations avec l'interface parallèle 8255 et Interruptions **19**
I Objectifs de TP 19
II Rappel 19
II.1 Rappel sur l'interface 8255 19
II.1.1 Brochage du 8255 19
II.1.2 Accès aux ports du 8255 19
II.1.3 Modes de fonctionnement de l'interface 8255 21
II.1.4 Connections du 8255 avec le microprocesseur 8086 21

II.2 Rappel sur les interruption . . . 22
II.2.1 Définition . . . 22
II.2.2 Interruptions du 8086 . . . 22
II.2.3 Traitement de demandes d'interruptions par le µp . . . 23
II.3 Préparation théorique . . . 24
III Manipulations . . . 25
III.1 Interface 8255 . . . 25
III.1.1 Contrôle de clignotement d'une LED à l'aide d'un programme . 25
III.1.2 Contrôle de clignotement d'une LED à l'aide des boutons poussoir 27
III.1.3 Contrôle d'un afficheur 7-segments . . . 28
III.2 Interruptions . . . 31

Partie II – Systèmes à microcontrôleur . . . 33

TP 3 : Contrôle des LED et Détection d'objets à l'aide de capteur Ultrason HC-SR04 . . . 34
I Objectifs de TP . . . 34
II Rappel . . . 34
II.1 Rappel sur la carte Arduino Uno . . . 34
II.1.1 Définition . . . 34
II.1.2 Présentation de la carte Arduino Uno . . . 34
II.2 Programmation de la carte Arduino Uno . . . 35
II.2.1 Environnement de développement (IDE) . . . 35
II.2.2 Programmation et configuration de la carte Arduino . . . 36
II.2.3 Langage Arduino . . . 37
II.3 Rappel sur le Capteur ultrason HC-SR04 . . . 39
II.3.1 Définition . . . 39
II.3.2 Principe de fonctionnement . . . 39
II.4 Signal PWM . . . 41
II.5 Préparation théorique . . . 42
III Manipulation . . . 42
III.1 Contrôle le clignotement d'une LED . . . 42
III.1.1 Contrôle de l'intensité lumineuse d'une LED . . . 43
III.1.2 Détection d'objets à l'aide de Capteur Ultrason HC-SR04 . . . 44

Partie I

Systèmes à microprocesseur

Introduction : Présentation générale du µp 8086 et de l'émulateur EMU8086

Le microprocesseur Intel 8086 qui est un microprocesseur classique souvent utilisé dans les milieux universitaires pour enseigner les concepts fondamentaux de la programmation en langage d'assemblage et de l'architecture des microprocesseurs. Il a été introduit en 1978 par Intel comme une extension 16 bits du microprocesseur 8 bits 8080 d'Intel, utilisant la technique de segmentation de la mémoire pour gérer efficacement la zone mémoire utilisable. C'est le premier microprocesseur de la famille Intel x86 (8086, 80186, 80286, 80386, 80486, Pentium, ...).

I Présentation générale du microprocesseur Intel 8086

I.1 Architecture externe du µp 8086

Le µp 8086 est un circuit intégré présenté sous forme d'un boîtier DIP à 40 broches comme le montre la figure 1. Il fonctionne soit en mode minimal (pour un système à un seul processeur), soit en mode maximal (pour un système à plusieurs processeurs). Le mode minimal est plus simple : le 8086 contrôle directement les bus. En revanche, le mode maximal est plus complexe : dans ce cas, un contrôleur de bus externe est utilisé pour gérer les signaux de contrôle et prendre en charge les différents microprocesseurs.

La sélection d'un mode se fait à l'aide de la broche $MN/\overline{MX}$ (broche 33). Parmi les 40 broches : le rôle de 32 broches (1-23 et 32-40) est le même pour les deux modes de fonctionnement du µp 8086, par contre le rôle des autres 8 broches (24-31) change suivant le mode de fonctionnement. Les 40 broches du µp 8086 peuvent être regroupées suivant leur fonction en 10 catégories, comme le montre le schéma fonctionnel de la figure 2.

I.2 Architecture interne du µp 8086

L'architecture interne de µp 8086 est constituée de deux unités indépendantes fonctionnant en parallèle (voir la figure 3) :

— EU (Execution Unit) ou l'unité d'exécution ;

— BIU (Bus Interface Unit) ou l'unité d'interface de bus.

Le tableau 1 présente brièvement le rôle de ces deux unités.

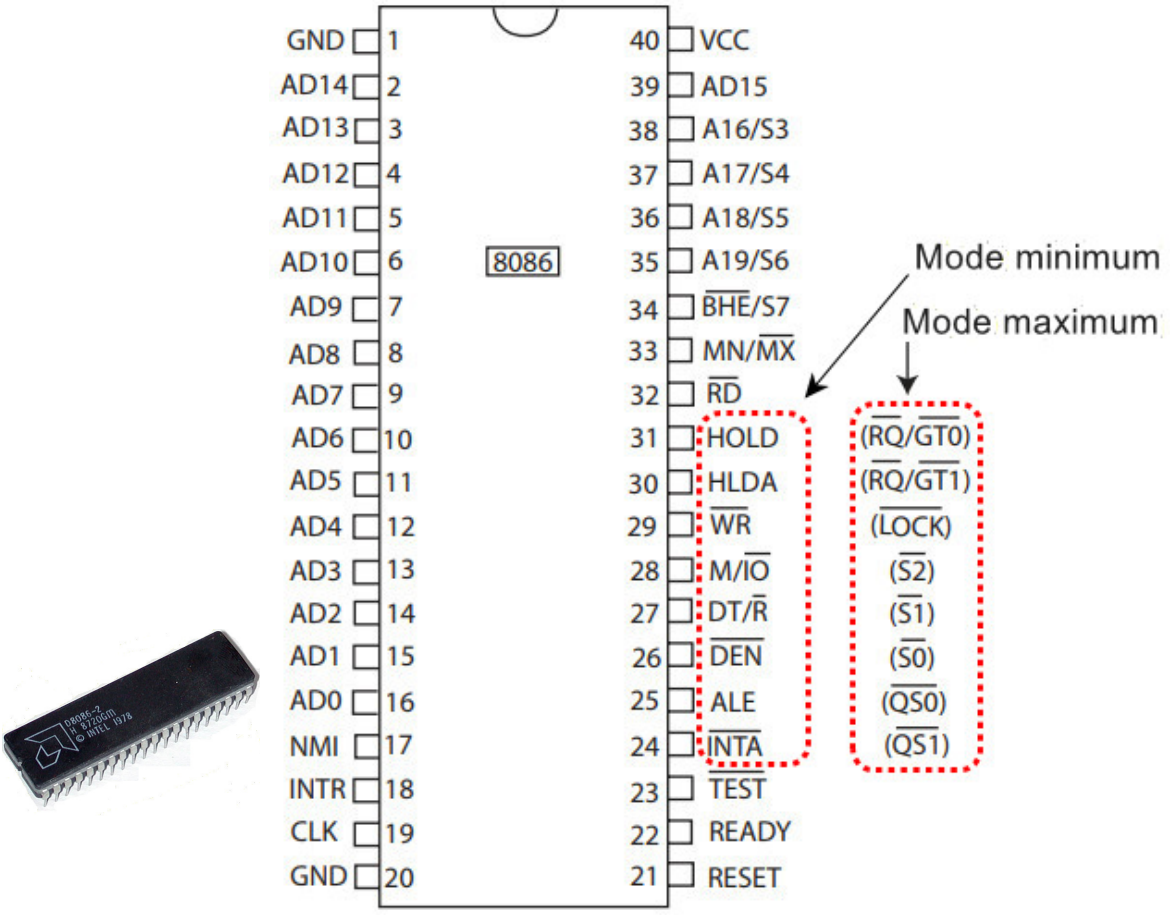


FIGURE 1 – Circuit intégré 8086 et affectation des broches du 8086 en mode min et max

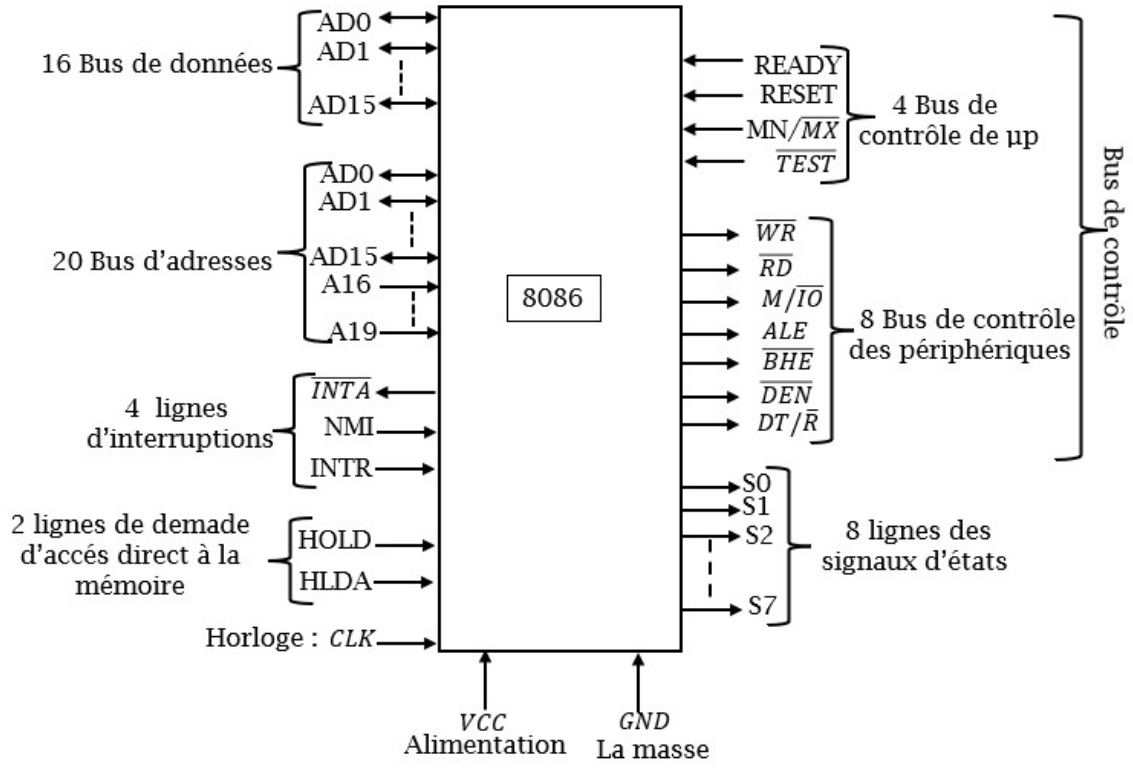


FIGURE 2 – Schéma fonctionnel du µp 8086.

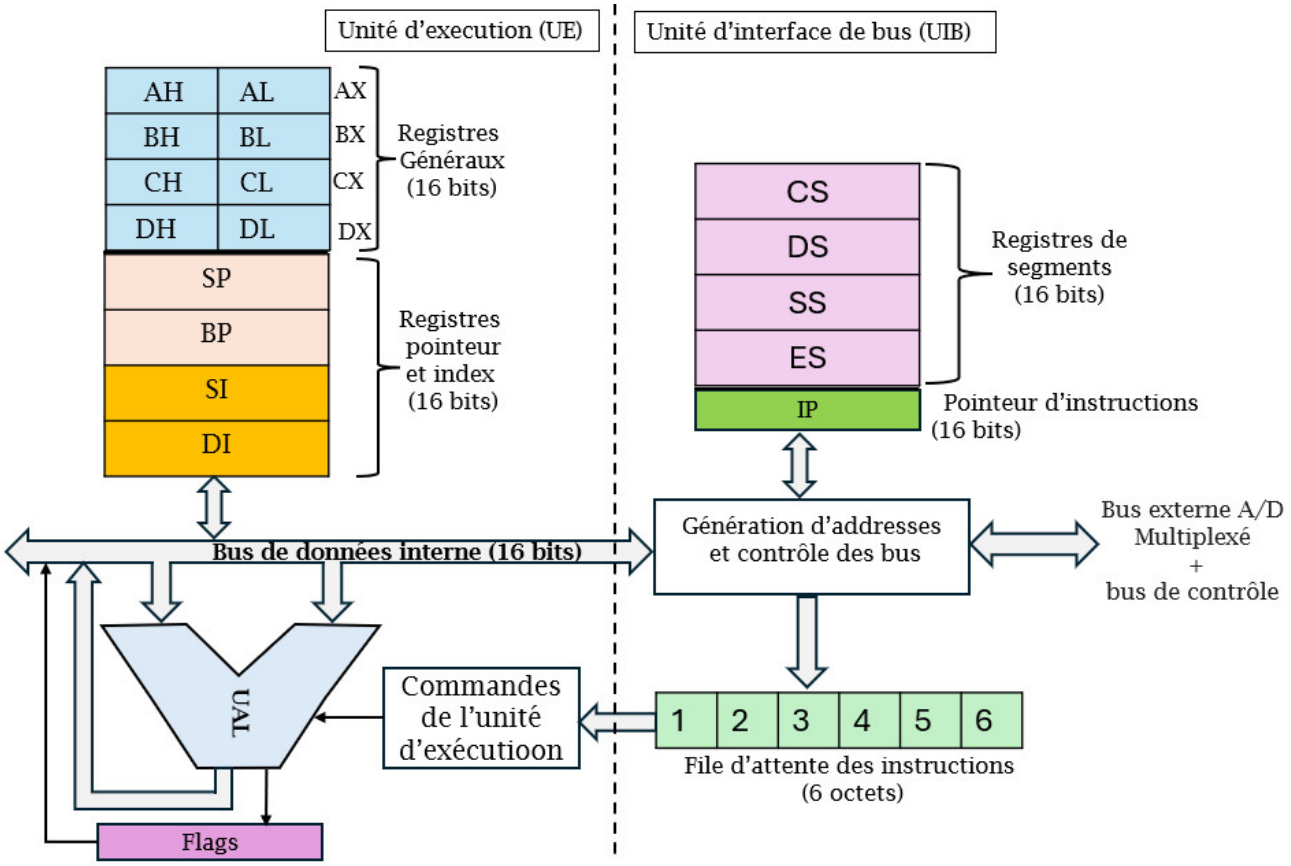


FIGURE 3 – Architecture interne de µp 8086.

Unité	Rôle
Unité d'interface de bus	Elle permet de : + Calculer les adresses physiques (adresses réelles) ; + Récupérer (Fetch en anglais) les instructions de la mémoire ; + Ranger les instructions dans une file d'attente ; + Établir les communications avec les périphériques du microprocesseur (Mémoires, Interfaces, d'E/S).
Unité d'exécution	Elle permet de : + Décoder les instructions contenues dans la file d'attente ; + Exécuter les instructions décodées.

TABLE 1 – Fonctions des unités EU et BIU du 8086.

I.2.1 Unité d'interface de bus

Comme le montre la figure 3, les éléments principaux de l'unité d'interface de bus sont :

— Registres de segments ;

— Pointeur d'instructions ;

— Bloc logique de génération d'adresses et de contrôle de bus ;

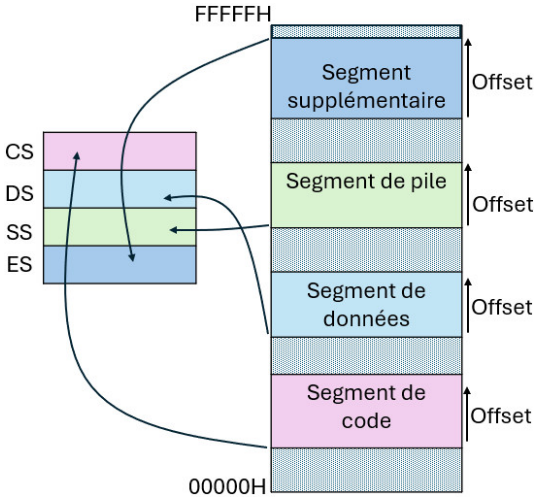
— File d'attente d'instructions.

I.2.1.1 Registres de segments : Dans le contexte du microprocesseur, un segment est une partie de l'espace mémoire, constituant donc une plage d'adresses dans la mémoire principale.

Le µp 8086 adresse une mémoire segmentée en quatre segments (divisée en quatre segments) : le *segment de code* inclut les instructions du programme, le *segment de données* inclut les données manipulées par le programme, le *segment de pile* inclut la pile de sauvegarde et le *segment supplémentaire* peut aussi inclure des données. Les adresses de ces quatre segments sont stockées dans les registres de segments suivants :

- **CS** (Code Segment) : Contient l'adresse du segment du code ;
- **DS** (Data Segment) : Contient l'adresse du segment de données ;
- **SS** (Stack Segment) : Contient l'adresse du segment de la pile ;
- **ES** (Extra Segment) : Contient l'adresse du segment supplémentaire.

Ces registres sont des registres de 16 bits.



I.2.1.2 Pointeur d'instruction : Le pointeur d'instruction (en anglais IP : Instruction Pointer) est un registre de 16 bits contenant l'adresse de la prochaine instruction à exécuter. L'adresse physique totale est calculée en combinant le contenu du registre CS et le contenu du pointeur IP.

I.2.1.3 Bloc logique de génération d'adresses et de contrôle des bus : Le rôle de ce bloc est la création des adresses pour accéder à la mémoire et à la gestion des signaux de contrôle pour coordonner les opérations des bus.

I.2.1.4 File d'attente d'instructions : La file d'attente des instructions de 6 octets permet de précharger l'instruction suivante dans la file pour améliorer les performances du microprocesseur. En effet, pour accélérer le processus de l'exécution, l'unité d'interface de bus du microprocesseur 8086 charge à l'avance les instructions suivantes dans cette file dont la taille est de 6 octets, ce qui correspond à la longueur maximale d'une instruction en langage machine pour cette architecture. Ce qui signifie que même l'instruction la plus longue peut être complètement chargée et stockée dans la file avant que le µp ne l'exécute, évitant ainsi des délais d'attente.

I.2.2 Unité d'exécution

Les composants principaux de cette unité sont :

- UAL (unité arithmétique et logique) ;
- Registres généraux ;
- Registre d'état ;
- Pointeurs de pile et index.

I.2.2.1 Unité arithmétique et logique : Comme son nom le suggère, cette unité peut effectuer deux types d'opérations :

- Les *opérations arithmétiques* telles que la sommation, la soustraction, la multiplication et la division ;
- Les *opérations logiques* à savoir OU, ET, OU _EXCLUSIF, de même les opérations de décalage et de rotation (arithmétique et logique).

La figure 4 montre le schéma général de l'unité arithmétique et logique (U.A.L), avec « n » est le nombre de bits. Pour le microprocesseur 8086, « n » peut être « 8 » ou « 16 ».

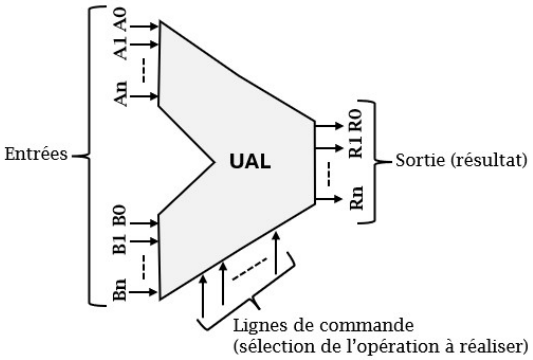


FIGURE 4 – Unité arithmétique et logique.

I.2.2.2 Registres généraux : Le µp 8086 contient 4 registres généraux sur 16 bits ou 8 registres sur 8 bits :

— **AX = (AH,AL)** (Accumulateur) ;
— **BX = (BH,BL)** (Base) ;
— **CX = (CH,CL)** (Compteur) ;
— **DX = (DH,DL)** (Données) .

Les alphabets X, H et L désignent respectivement mot (16 bits), octet du poids fort (8 bits) et octet du poids faible (8 bits).
Exemple : AX est le registre sur 16 bit (mot), AL est le registre d'octet de poids faible, AH est le registre d'octet de poids fort.

Ces registres sont employés pour stocker temporairement des données pendant l'exécution des programmes, pour les calculs et le stockage d'adresses, et pour la gestion du flux d'exécution des instructions. Malgré que ces registres sont généraux, ils peuvent être employés pour réaliser des tâches particulières en dehors des opérations arithmétiques et logiques classiques. En effet :

- AX, AH, AL : sont employés lors des opérations arithmétiques et logiques (accumulateurs) ;
- BX : est utilisé pour le calcul des adresses ;
- CX et CL : sont utilisés comme des compteurs de boucles ;
- DX : est utilisé pour stocker les adresses des ports d'entrée/sortie et utilisé également d'extension à AX, dans le cas des opérations arithmétiques (opération sur 32 bits).

I.2.2.3 Registres de pointeurs et d'index : Ces registres sont aussi connus sous le nom des registres d'adresse. Ils sont sur 16 bits et employés pour gérer les adresses mémoire, permettant au microprocesseur de localiser et d'accéder aux instructions et aux données en mémoire.

— *Pointeurs* : ils sont utilisés pour la gestion de la mémoire, en particulier de la pile. On distingue entre :
  — **SP** (Stack Pointer en anglais ou pointeur de pile) : contient l'adresse de l'élément supérieur de la pile, permettant de sauvegarder des données en cours d'exécution d'un programme ;
  — **BP** (Base Pointer en anglais ou pointeur de base en français) : utilisé pour accéder aux données dans la pile.

— *Index* : ces registres sont utilisés fréquemment lors des transferts de chaînes de caractères et de tableaux entre deux zones mémoire, car ils permettent de localiser respectivement la source et la destination des données à déplacer ou à traiter. On distingue entre :

— **SI** (Source Index) : sert à pointer vers la source des données lors d'une opération, par exemple pour lire des données depuis une adresse mémoire.

— **DI** (Destination Index) : sert à pointer vers la destination des données, c'est-à-dire l'adresse où les données doivent être écrites.

I.2.2.4 Registre d'état : Le *registre d'état* (aussi connu sous le nom de registre des drapeaux, ou flags en anglais) est un registre sur 16 bits qui indique l'état du microprocesseur suite à l'exécution d'une opération. Chacun des 16 bits (drapeaux) peut être mis à 0 ou 1 pour signaler une condition particulière. Ils sont utilisés par le microprocesseur pour prendre des décisions, comme dans le cas des sauts conditionnels et les boucles. Les drapeaux de ce registre peuvent être divisés en *drapeaux conditionnels* et *drapeaux de contrôle* (figure 5).

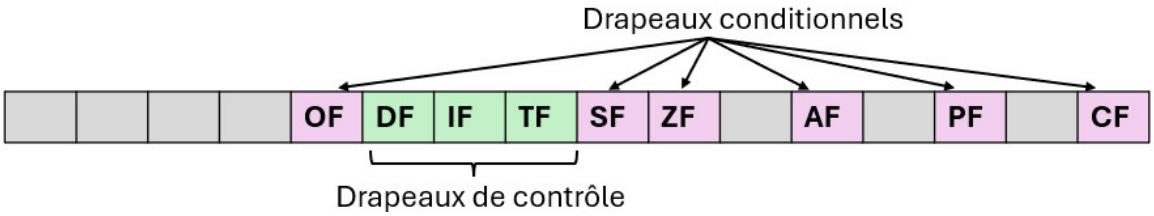


FIGURE 5 – Registre d'état.

- **Drapeaux conditionnels :** indiquent le résultat d'une opération arithmétique ou logique. Cette catégorie de drapeaux ainsi que leur rôle sont comme suit :

— **CF** : c'est le bit de retenue (Carry en anglais). $CF = 1$, si une retenue est produite lors d'une addition des nombres non-signés ou si un emprunt est nécessaire lors d'une soustraction.

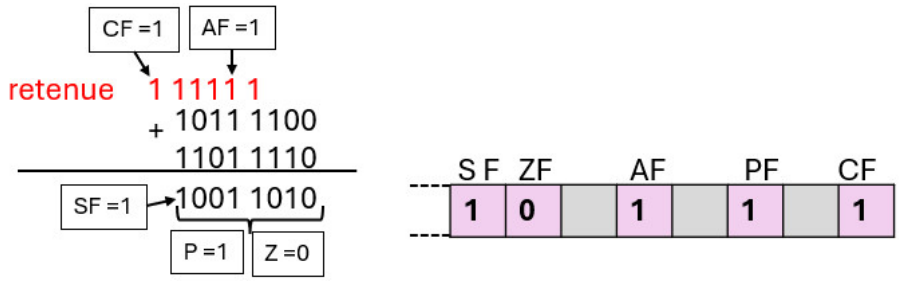
— **PF** : c'est le bit de parité (Parity en anglais). $PF = 1$, si le résultat d'une opération contient un nombre pair de bits à 1 (parité paire), et à 0 si le nombre de bits à 1 est impair (parité impaire).

— **AF** : représente le bit de la retenue auxiliaire (Auxillary Carry). $AC = 1$, lorsqu'une opération arithmétique génère une retenue ou un emprunt entre le demi-octet (nibble) de poids faible et le demi-octet de poids fort (les 4 bits de droite et les 4 bits de gauche). Cet indicateur est utilisé lors des opérations 8 bits.

— **ZF** : c'est un indicateur de la nullité (Zero). $ZF = 1$, si le résultat obtenu est égal à 0.

— **SF** : c'est un bit de signe (Sign). Ce drapeau est employé lors de manipulation des nombres signés. $SF = 1$, si le résultat est négatif (c.à.d., le bit le plus significatif est 1), sinon $SF = 0$.

— **OF** : c'est le bit de débordement (Overflow). $OF = 1$, si le résultat d'une opération sur les nombres signés dépasse la capacité du registre (par exemple, si un grand nombre positif est additionné à un autre grand nombre positif, ce qui donne un résultat négatif).



Exemple : Quelles sont les valeurs des drapeaux conditionnels lors de l'addition de deux nombres suivants 10111100 et 11011110 ?

- **Drapeaux de contrôle :** sont utilisés pour contrôler le fonctionnement du microprocesseur. On distingue trois principaux drapeaux de contrôle :
  - **TF** : représente le bit de mode pas-à-pas (Trap en anglais). Si $TF = 1$, le microprocesseur passe en mode pas-à-pas, exécutant une instruction à la fois, puis déclenche une interruption.
  - **IF** : ce bit indique si le microprocesseur va répondre aux interruptions ou non (Interrupt). Si $IF = 1$, le drapeau est activé, le processeur répond aux interruptions du périphérique d'E/S et les programmes en cours d'exécution seront interrompus momentanément ; sinon, il les ignore.
  - **DF** : c'est un indicateur de direction (Direction). Il indique la direction de l'incrémentation ou de la décrémentation des pointeurs de chaîne lors des opérations de chaînes (comme dans le cas des instructions MOVSB ou LODSB). Si $DF = 0$, la chaîne sera lue dans l'ordre croissant. Si $DF = 1$, la chaîne sera lue dans l'ordre décroissant.

II Présentation de l'émulateur 8086 : Emu8086

Emulateur 8086 (EMU8086) est un logiciel qui permet de reproduire le fonctionnement du microprocesseur 8086. Il permet de développer et d'exécuter des programmes en langage assembleur. Lien de téléchargement : https ://emu8086-microprocessoremulator.fr.softonic.com/

Voici les étapes à suivre pour simuler un programmer sous EMU8086 :

Etape 1: Après l'installation de EMU8086, puis le double clic sur son icône (figure 6.a), la façade montrée par la figure 6.b est affichée. Comme le montre cette figure, nous pouvons choisir un nouveau fichier "new", exemples de codes "code examples", démarrage rapide "quick start ou fichier récemment créés "recent files".



(a) Icône de l'émulateur du 8086 : emu8086

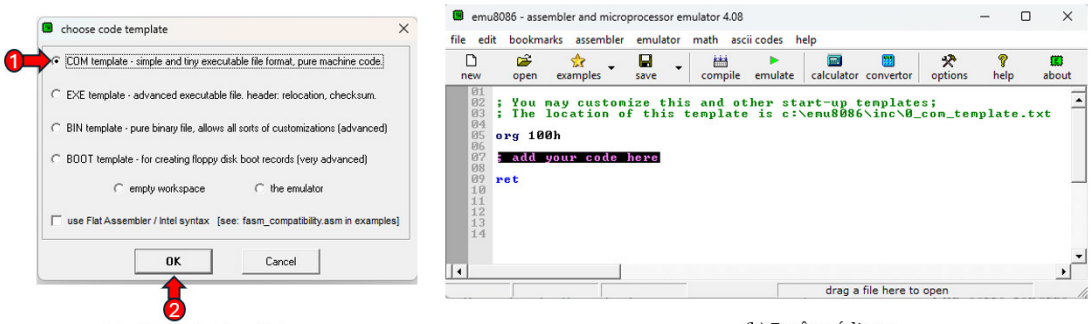
(b) Façade affichée après double clic sur l'icône EMU8086

FIGURE 6 – Icône et la première façade de l'émulateur 8086.

Etape 2: - Pour simuler un programmer spécifique clic sur "new". La fenêtre « choose code template » sera immédiatement affichée (voir la figure 7.a).

Sur cette fenêtre, on peut choisir entre quatre templates ".COM", ".EXE", ".BIN" et ".BOOT". Le template .COM crée de petits exécutables de moins de 64 Ko, le .EXE gère des programmes plus complexes avec segments, le .BIN est du code machine brut (produit un fichier binaire brut) et .BOOT est un secteur de démarrage (512 octets).

- Parmi ces quatre templates choisir le template .COM. Après le clic sur "ok", la fenêtre de l'éditeur est affichée (7.b) .



(a) Choix de Template

(b) Fenêtre éditeur

FIGURE 7 – Choix de Template et éditeur de texte.

Etape 3: - Écrire le code dans la zone où il y a le commentaire " ; add your code here". On considère le code suivant :

mov ax, 15h ; charger le registre ax avec 15h
mov bx, 25h ; charger le registre bx avec 25h
add ax, bx ; ajouter le contenu du registre bx au contenu du registre ax

mov w.[0300h], ax ; stocker le contenu du registre ax dans deux cases mémoires (2 octects) d'adresses 03000h et 03001h.
- Cliquer sur l'icône "save" pour sauvegarder le code sous le nom « mycode.asm ».

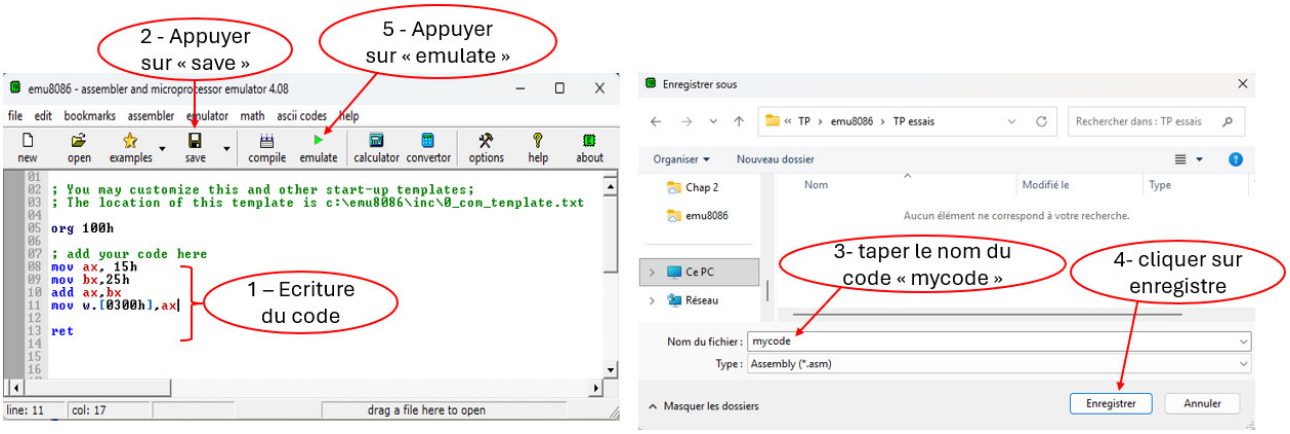
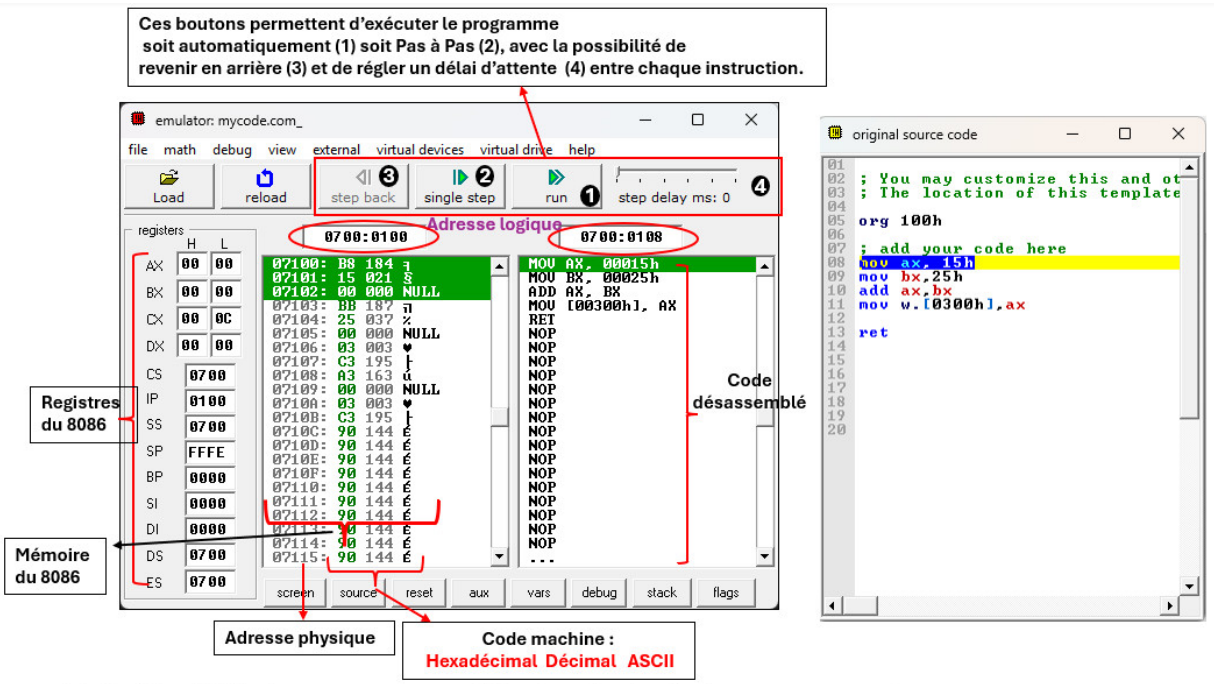


FIGURE 8 – Écriture et enregistrement du code.

Etape 4: Compilation du code : appuyer sur l'icône « emulate » pour compiler/linker et émuler le code (voir figure 8). Suite à cette action, deux fenêtres sont affichées :
- La fenêtre « original source code » contient le code tel que vous l'avez écrit. La ligne surlignée en jaune est la prochaine instruction qui va être exécutée (figure 9.b).
- La fenêtre 8086 microprocesseur emulator peut être répartie en deux zones (figure 9.a) : les registres et mémoire.



(a) Fenêtre 8086 microprocesseur emulator (avant exécution)

(b) Fenêtre « original source code »

FIGURE 9 – Fenêtres émulateur du 8086 microprocesseur et "original source code".

La figure 10 montre la fenêtre émulateur du 8086 après le clic sur l'exécution automatique "run".

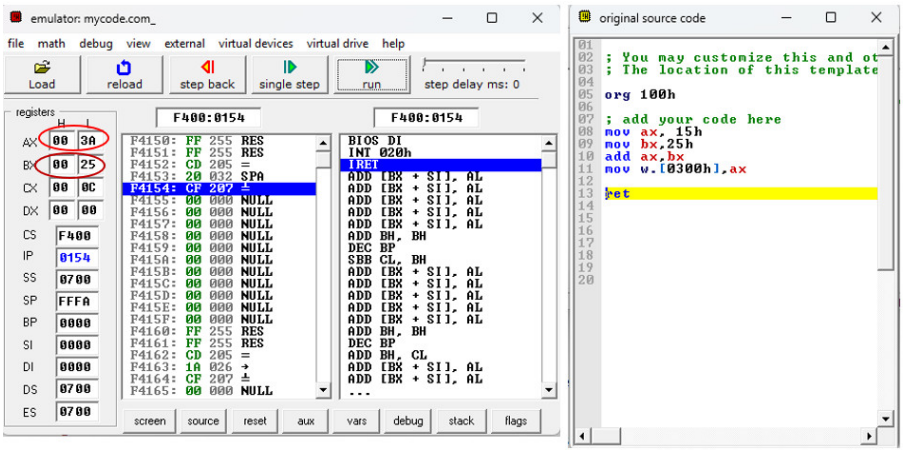


FIGURE 10 – Fenêtre émulateur du 8086 après exécution automatique.

Figures 11-15 montrent la fenêtre émulateur du 8086 après l'exécution du programme avec le mode "single step".

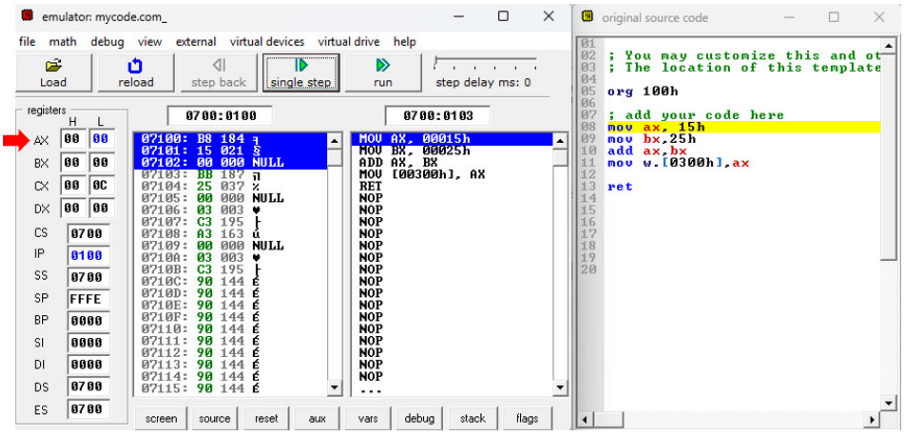


FIGURE 11 – Fenêtre émulateur du 8086 avant exécution de *mov ax,15h*.

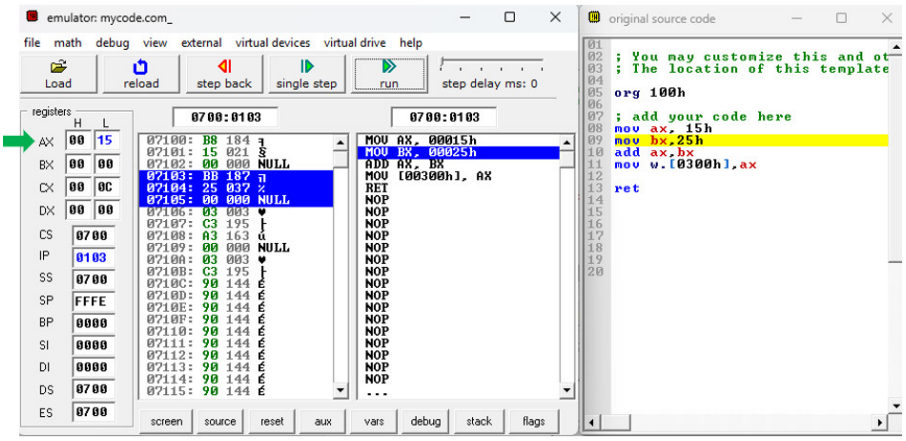


FIGURE 12 – Fenêtre émulateur du 8086 après exécution de *mov ax,15h*.

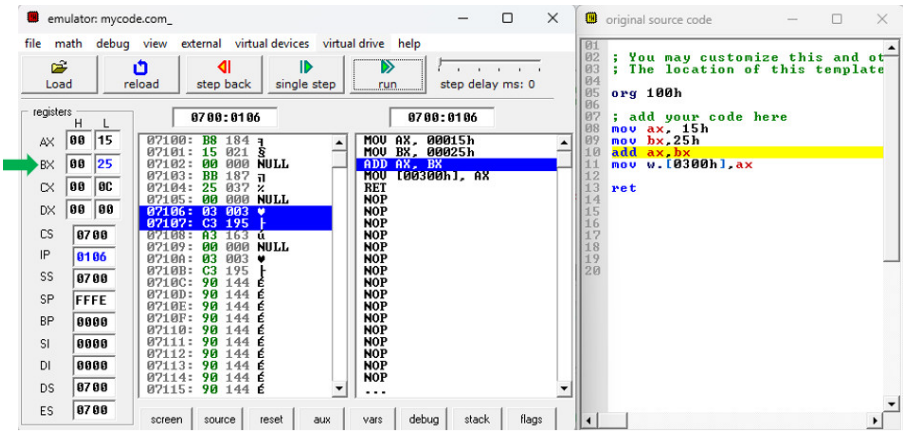


FIGURE 13 – Fenêtre émulateur du 8086 après exécution de *mov bx,25h*.

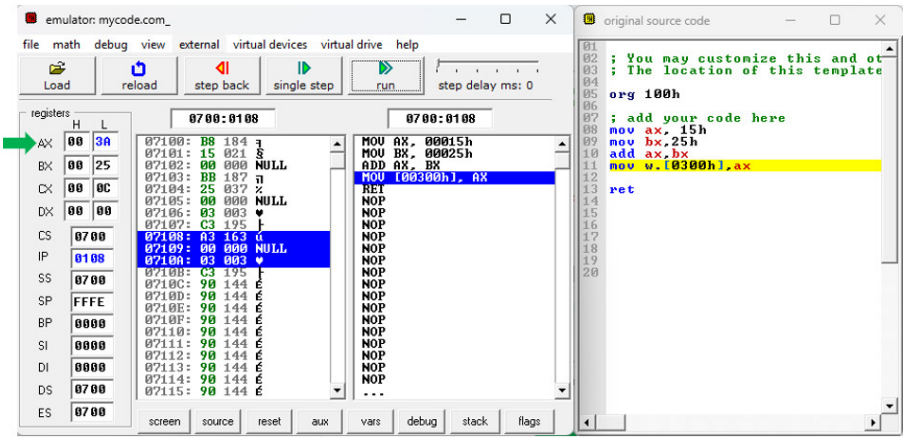


FIGURE 14 – Fenêtre émulateur du 8086 après exécution de *add ax,bx*.

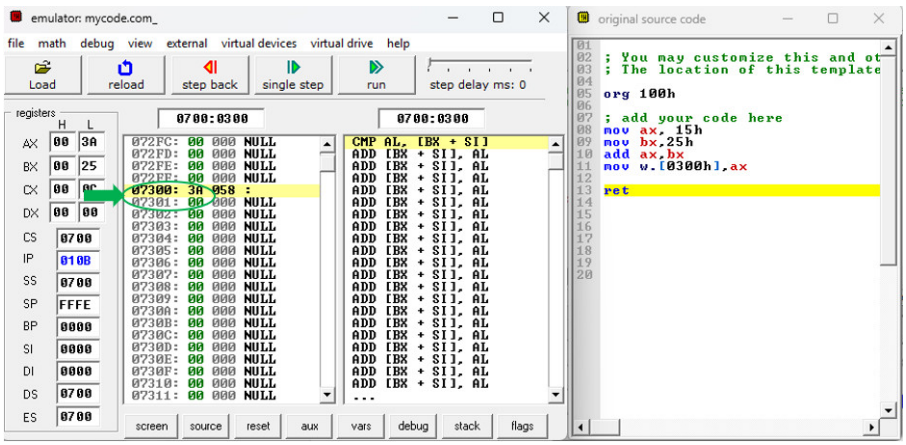


FIGURE 15 – Fenêtre émulateur du 8086 après exécution de *mov w.[0300h],ax*.

TP 1 : Prise en main de l'émulateur 8086 et programmation en assembleur

I Objectifs de TP

A l'issue de ce TP, l'étudiant doit :maîtriser l'utilisation de l'émulateur 8086 ; savoir manipuler les registres et la mémoire du µp 8086 ; être capable de concevoir des programmes en langage assembleur permettant de réaliser des fonctions spécifiques.

II Préparation théorique

1. Classer les mots suivants en directives instructions et autres.

- mov
- cmp
- encore :
- push
- ax
- org 100h
- dd
- ah
- Start :
- jne
- Assume
- Ends

Instructions	Directives	autres

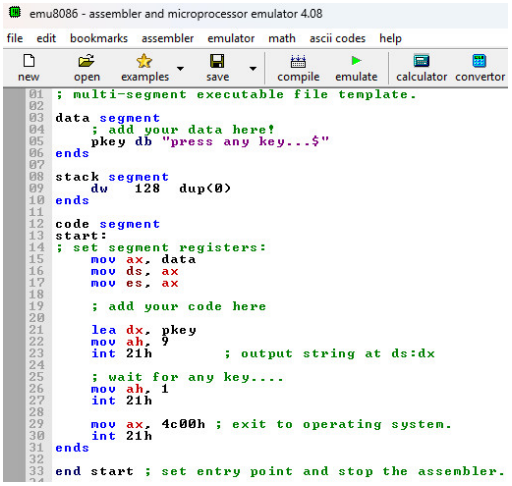
2. Compléter le tableau suivant en précisant le mode d'adressage utilisé.

Instructions	Modes d'adressage	Instructions	Modes d'adressage
MOV AX, 0005H		MOV AL, [BX]	
MOV AX, BX		MOV AL, [BP]	
MOV BL,[1300H]		MOV AL, [SI]	
MOV BYTE PTR [2100H], 55H		MOV [DI], BX	

3. Quelle est la valeur de chacune des variables suivantes :

- var1 DW 10 ⟶ var1=
- var2 DW 10H ⟶ var2=
- nb1 DD ? ⟶ nb1=
- vect DW 7 DUP (0) ⟶vect =
- vect DW 7 DUP (0,1) ⟶ vect =
- vect DW 7 DUP (?)⟶ vect =
- four EQU 4*3 ⟶ four =
- Chain DB 'This is a message' ⟶ Chain =

4. Quels sont les principaux blocs du programme suivant ?



-.....

-.....

-.....

III Manipulation

III.1 Modes d'adressage

1. Lancer l'environnement Emu8086 et appuyer sur l'onglet New. Puis choisissez .COM template.
2. Taper (sans les commentaires, texte après ;)et sauvegarder le programme suivant sous le nom tp1_mod_add

org 100h ;	Adresse de départ du code/des données
mov ax, 15h ;	Charger ax avec 15h
mov bx,2502h ;	Charger bx avec 2502h
mov w.[0300h],ax ;	Stocke le contenu de ax (mot 0015h) à l'adresse mémoire DS :0300h
mov w.[0302h],bx ;	Stocke le contenu de bx (mot 2502h) à l'adresse mémoire DS :0302h
mov [0304h],35h ;	Stocke l'octet 35h à l'adresse DS :0304h
mov bx,0300h ;	Charger bx avec 0300h (pointeur vers la zone mémoire)
mov cx,[bx] ;	Charger cx avec le contenu de la zone mémoire DS :bx (DS :0300h)
mov dx,[bx+2] ;	Charge dx avec DS :bx+2 (DS :0302)
mov ax,[bx]+4 ;	Charger ax avec (le contenu mémoire DS :0300h)+ 4
mov si,bx ;	Charger le registre SI avec 0300h
mov di,[si] ;	Charger DI avec le contenu du mot à DS :0300h
ret ;	Fin du programme .COM

3. Émulez ce code en cliquant sur « emulate ».
4. Exécutez ce code en appuyant sur "run" ?
   (a) Que remarquez-vous concernant le contenu des différents registres ?

   . . . . . .

   . . . . . .

   (b) Que remarquez-vous concernant la mémoire ?

   . . . . . .

. . . . . .

5. Exécuter ce code en mode pas à pas en appuyant sur "single run" et complétez le tableau suivant.

Instruction	Code machine (Hexa)	Adresse logique	Adresse physique	Registre ou cases mémoires modifiées	Commentaire (opération + mode d'adressage)
org 100h					
mov ax, 15h					
mov bx,2502h					
mov w.[0300h],ax					
mov w.[03002h],bx					
mov [03004h],35h					
mov bx,0300h					
mov cx,[bx]					
mov dx,[bx+2]					
mov ax,[bx]+4					
mov si,bx					
mov di,[si]					
ret					

III.2 Opérations arithmétiques et logiques

III.2.1 Opérations arithmétiques

Écrire et simuler un programme en assembleur qui permet de faire la somme, la soustraction, le produit et la multiplication de deux nombres de 16 bits, nommée respectivement var1 (=1525h) et var2 (=1000h). Les résultats doivent être sauvegardés respectivement dans la mémoire à partir de l'adresse logique 100h :300h.
Voici l'algorithme à suivre :

(i) Lancer l'environnement Emu8086 et appuyer sur l'onglet New. Puis choisissez .COM template.

(ii) Déclarer les variables var1 et var2 ;

(iii) Pour chaque opération arithmétique :

  (a) Charger var1 dans ax ;

(b) Réaliser l'opération arithmétique entre le contenu de ax et var2 ;

(c) Stocker le résultat dans la mémoire ; (pour la division stocker le quotient puis le reste).

(iv) Exécuter le code que vous avez développé en mode pas à pas en appuyant sur "single run".

(a) Visualiser le contenu du registre d'états (flags) ;

(b) Conclure.

Rappel :

ADD Opérande1,Opérande2 ; l'opération à effectuer est opérande1 ←opérande1 + opérande2	**SUB Opérande1,Opérande2** ; l'opération à effectuer est opérande1 ←opérande1 - opérande2
MUL opérande ; Multiplier le contenu de l'opérande (Registre/ Mémoire) avec le contenu du registre AL ou AX. - Lorsque l'opérande est un octet (8 bits) : AX = AL * opérande. - Lorsque l'opérande est un mot (16 bits) : (DX : AX) = AX *opérande.	**DIV Opérande ;** Division du contenu du registre AX ou (DX :AX) sur le contenu de l'opérande (Registre/ Mémoire). - Lorsque l'opérande est un octet : AL = AX / opérande, AH = reste - Lorsque l'opérande est un mot : AX = (DX : AX) / opérande, DX = reste

III.2.2 Opérations logiques

Écrire et simuler un programme en assembleur qui permet de faire les fonctions logiques suivantes entre deux nombres de 8 bits, nommés respectivement var1 (=11001100b) et var2(=00110011b) : Et-logique, ou-logique, Ou_exclusif, complément à 1 et complément à deux. Les résultats doivent être sauvegardés respectivement dans les variables result1, result2, result3, result4 et result5.

Voici l'algorithme à suivre :

(i) Lancer l'environnement Emu8086 et appuyer sur l'onglet New. Puis choisissez .COM template.

(ii) Déclarer les variables var1, var2, result1, result2, result3, result4 et result5.

(iii) Pour chaque opération logique :

(a) Charger var1 dans AL ;

(b) Réaliser l'opération arithmétique entre le contenu de AL et var2 ;

(c) Stocker le résultat dans la variable correspondante ;

(iv) Exécuter le code que vous avez développé en mode pas à pas en appuyant sur "single run".

(a) Visualiser le contenu du registre d'états (flags) ;

(b) Conclure.

Rappel :

AND opérande1, opérande2 ; opérande1←opérande1 ET opérande2 ; opérande2	**OR opérande1, opérande2 ;** opérande1←opérande1 OU opérande2 ;
XOR opérande1, opérande2 ; opérande1←opérande1 $\oplus$ opérande2	- **NOT opérande1** ; opérande1←$\overline{oprande1}$ - **NEG opérande1** ; opérande1←$\overline{oprande1}$+1

III.3 Boucles et instructions de branchement : Opérations sur les tableaux

Écrire un programme en assembleur qui permet de trouver le maximum et le minimum d'un tableau. Par exemple, on considère un tableau de 8 octets défini comme suit TABLE={15, 20, 10, 11, 2,35,60,30}.
Voici l'algorithme à suivre :

(i) Définir le segment de données par l'instruction de définition de SEGMENT
   (a) Initialiser le tableau, la variable max, la variable min et la variable "taille" (la taille de tableau) ;

(ii) Définir le segment de code.
   (a) Les étiquettes (les noms) des segments de code et de données sont attribuées aux segments CS et DS par la directive ASSUME.
   (b) L'emplacement de mémoire qui est défini par le nom Data est chargé dans le registre de segment DS via le registre AX. Le registre AX est ici utilisé, car les registres de segment ne peuvent pas être chargés avec les données immédiates.

(iii) Initialiser le registre SI avec l'adresse du tableau ;

(iv) Initialiser MAX avec le 1er élément du tableau ;

(v) Initialiser MIN avec le 1er élément du tableau ;

(vi) Initialiser le compteur de boucle (CX) avec la taille du tableau - 1, car on a déjà pris le 1er élément.

(vii) Boucler sur les éléments restants :
   (a) Comparer l'élément courant avec MAX : Si supérieur, mettre à jour MAX.
   (b) Comparer l'élément courant avec MIN : Si inférieur, mettre à jour MIN.

(viii) Continuer jusqu'à la fin du tableau.

(ix) Exécuter le code que vous avez développé en mode pas à pas en appuyant sur "single run".
   (a) Quelles sont les adresses logique et physique des variables min et max.
   (b) Visualiser l'évolution du contenu de ces deux variables.

Rappel :

DATA SEGMENT ; Définition de segment DATA ..... ; déclarer les variables DATA ENDS CODE SEGMENT ASSUME DS : DATA, CS : CODE START : MOV AX ,DATA ; Initialiser le registre DS MOV DS,AX ..... ; Ajouter votre code CODE ENDS ; END START END	- **INC opérande** : incrémentation d'une unité le contenu d'un opérande (Exemple INC AX) - **DEC opérande** : décrémentation d'une unité le contenu d'un opérande (DEC AX) - **CMP opérande 1, opérande2** ; compare les deux opérandes en soustrayant le second opérande du premier ($op1 - op2$) - **JBE label** ; (Jump if Below or Equal) Saut si inférieur ou égal, $CF = 1$ et $ZF = 1$ opérande2 - **JAE label** ; (Jump if Above or Equal) Saut si supérieur ou égal, $CF = 0$

III.4 Instructions de décalage et rotation

A l'aide des instructions de décalage (SHL, SHR) et de rotation (ROL, ROR), écrire un programme qui permet d'effectuer :

1. La division et la multiplication du nombre 20 par 4,
2. Permutation des demi-octets (nibbles) du nombre 11110000.

Rappel :

SHL (Shift Left) ; Multiplie par 2 pour chaque décalage. Exemple : SHL, AL,n	**SHR (Shift Right)** : Divise un nombre par 2 pour chaque décalage, Exemple : SHR, AL,n
ROL (Rotate Left) ; Le bit de poids fort (MSB) passe au bit de poids faible (LSB) et dans le CF.Exemple : ROL, AL,n	**ROR (Rotate Right)** ; Le bit de poids faible passe au bit de poids fort et dans le CF, Exemple : ROR, AL,n

Remarques :

- Si le nombre n est supérieur à 1, ce nombre doit être chargé dans le registre CL (ou CX) avant l'exécution de l'instruction ;
- Dans le cas où $n > 1$ seul le dernier bit qui sort est conservé dans le drapeau CF.

Exemple : décalage par 3 positions ($n = 3$)
MOV AL, 11001011B ; AL ←11001011B
MOV CL,3 ;
SHR AL,CL ; Décalage logique de 3 positions à droite : AL ←00011001 et CF ←0

TP 2 : Manipulations avec l'interface parallèle 8255 et Interruptions

I Objectifs de TP

A l'issue de ce TP, l'étudiant doit :Maitriser la communication entre le microprocesseur 8086 et l'interface parallèle 8255 à travers un ensemble d'exemples tels que le contrôle de clignotement d'une LED à l'aide d'un programme, contrôle de clignotement d'une LED à l'aide des boutons poussoir, et contrôle d'un afficheur 7-segments. Maitriser l'utilisation du logiciel Proteus pour la conception et le contrôle en assembleur des systèmes électroniques. Savoir manipuler les interruptions pour lire et afficher les données.

II Rappel

II.1 Rappel sur l'interface 8255

II.1.1 Brochage du 8255

L'interface 8255 est une interface parallèle programmable de 40 broches (voir figure 16) permettant de transférer des données en parallèle du microprocesseur vers des périphériques et inversement. Elle peut être configurée en entrée et/ou en sortie par programme.
Les différentes broches peuvent être regroupées comme suit :

- Les ports A, B et C. Ces ports sont regroupés en deux groupes : groupe A constitué par le port A et le port C haut et groupe B constitué par le port B et le port C bas ;
- Les lignes de contrôle : $\overline{RD}$ et $\overline{WR}$ sont les signaux de lecture/écriture ; A_0 et A_1 sont les bus d'adresse ; $\overline{CS}$ c'est le signal de sélection de l'interface ; $RESET$ c'est le signal de réinitialisation ;
- Le bus de données ;
- Les entrées d'alimentation Vcc et GND.

II.1.2 Accès aux ports du 8255

L'interface 8255 dispose de quatre registres :

- *Trois registres de données* servant de recevoir les données présentées sur les ports A, B et C lors de la lecture ou de l'écriture. Chaque port est associé à un registre. L'accès à un port donné se fait en accédant à son registre.

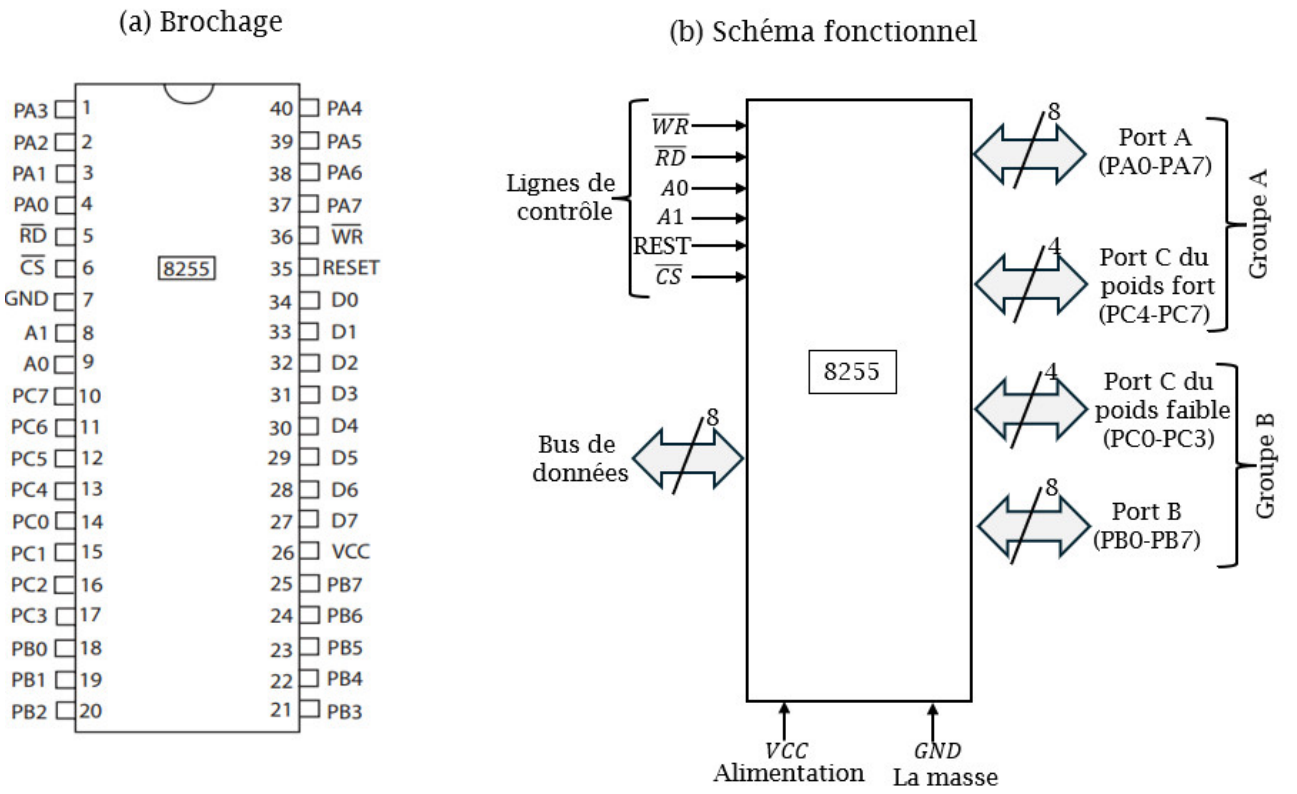


FIGURE 16 – Architecture externe de l'interface parallèle 8255.

- *Un registre de contrôle* permettant de configurer des ports A, B et C en entrées et/ou en sorties. Ce registre est accessible uniquement en écriture, c.à.d., sa lecture n'est pas autorisée.

La sélection de l'un de ces registres (ports) se fait à l'aide de deux bits d'adresse $A0$ et $A1$ selon la table de vérité 2. Plus en détail, l'adressage des différents ports en entrée et/ou sortie se fait comme le montre le tableau 3.

A_0	A_1	Registre (port)
0	0	Port A
0	1	Port B
0	0	Port C
1	1	Registre de contrôle

TABLE 2 – Sélection d'un port.

A_0	A_1	$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	Opération de lecture (Read))
0	0	0	1	0	Lecture du port A
0	1	0	1	0	Lecture du Port B
1	0	0	1	0	Lecture du Port C
					Opération de l'écriture (Write)
0	0	0	1	0	Écriture du port A
0	1	0	1	0	Écriture du Port B
1	0	0	1	0	Écriture du Port C
1	1	1	0	0	Écriture du registre de commande
					L'interface 8255 hors fonctionnement
X	X	X	X	1	Bus de données troisième état
X	X	1	1	0	Bus de données troisième état
1	1	0	1	0	Impossible

TABLE 3 – Sélection et manipulation des ports.

II.1.3 Modes de fonctionnement de l'interface 8255

Suivant le contenu de registre de commande, les ports peuvent être configurés en entrées ou en sorties. De plus, on distingue trois modes de fonctionnement de 8255 :

— **Mode 0** : c'est le mode le plus simple. Dans ce mode, les ports sont configurés en entrée/sortie de base. Les données écrites dans les registres correspondants sont mémorisées sur les lignes de sorties ; l'état des lignes d'entrées est recopié dans les registres correspondants et n'est pas mémorisé. Il n'utilise pas les signaux de contrôle de flux (handshaking). Il est idéal pour des périphériques simples (comme un afficheur simple).

— **Mode 1** : appelé aussi mode d'entrée/sortie échantillonnée (ou strobée). Dans ce mode, le microprocesseur contrôle directement le timing des transferts de données avec les périphériques, utilisant les signaux de contrôle (handshaking) pour gérer le flux de données, comme Strobe (STROB) et Input Buffer Full (IBF). Il permet donc de synchroniser les transferts entre le microprocesseur et le périphérique.

— **Mode 2** : appelé aussi mode de bus bidirectionnel. Il permet une communication bidirectionnelle sur le Port A avec des signaux de contrôle provenant du Port C (PC3-PC7). Il est utilisé pour l'échange de données entre périphériques (comme un contrôleur de disquette) ou avec un autre microprocesseur, supportant les interruptions et utilisant des bascules pour mémoriser les données entrantes et sortantes.

La figure 17 montre la configuration des ports ainsi que le choix des modes selon le contenu du registre de commande. Le rôle des bits D0-D6 change suivant le contenu du bit D7. D7=1 : Mode E/S, les bits D6-D0 sont utilisés pour configurer les Ports A, B et C en entrée/sortie. D7=0 : Mode BSR (Bit Set/Reset) les bits D6-D0 sont utilisés pour manipuler individuellement les bits du Port C.

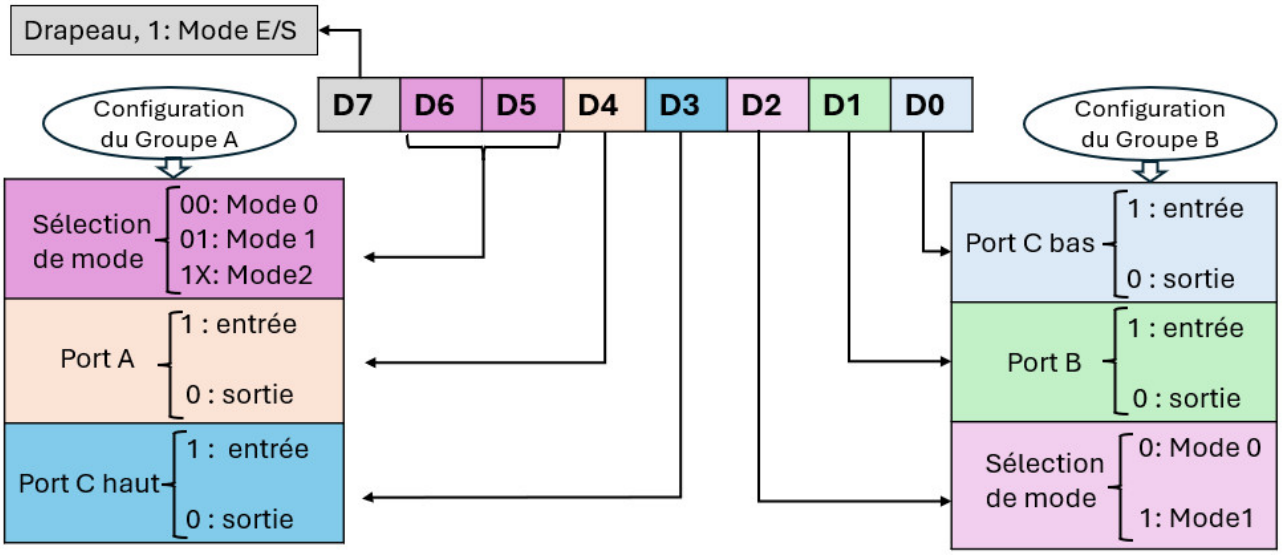


FIGURE 17 – Configuration des ports ainsi que le choix des modes selon le contenu du registre de commande.

II.1.4 Connections du 8255 avec le microprocesseur 8086

Comme nous avons vu, le bus de données du microprocesseur 8086 est sur 16 bits, cependant le bus de données de l'interface 8255 est sur 8 bits. On peut donc connecter le bus de données

du 8255 sur les lignes de données de poids fort du 8086 (D8-D15) ou sur celles de poids faible (D0-D7).

Comme nous avons vu dans le chapitre 2, une donnée de 8 bits est envoyée ou reçue par le microprocesseur sur les lignes :

— D0-D7 si l'adresse de la donnée est paire, validation par A0 =0 ;

— D8-D15 si l'adresse de la donnée est impaire, validation par $\overline{BHE} = 0$ (bus high enable) ;

L'un de ces deux signaux peut don être utilisé conjointement avec la sortie d'un décodeur d'adresses pour sélectionner l'interface 8255 (voir la figure 18). On utilise le signal $A0$ dans le cas où les adresses de 8255 sont dans la banque pair (des adresses paires) et $\overline{BHE}$ s'ils sont dans la banque impaire (des adresses impaires). Les bits $A1$ et $A2$ sont utilisés pour sélectionner un registre interne de 8255. A3-A15 sont employés par le décodeur pour sélectionner l'adresse de base de 8255.

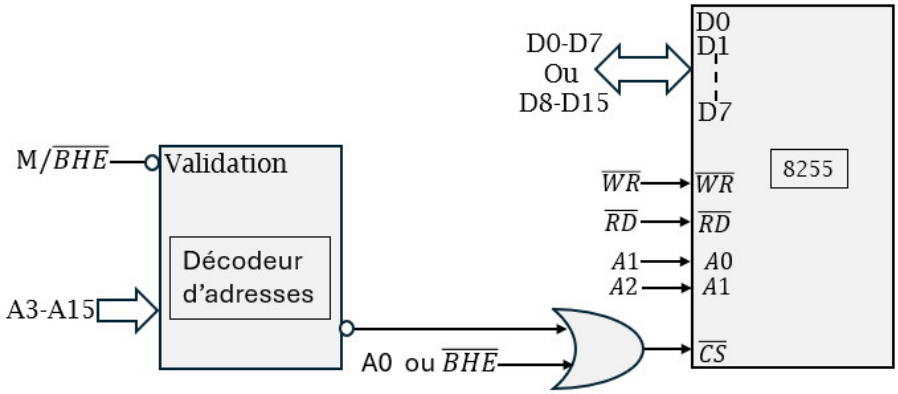


FIGURE 18 – Circuit de sélection de l'interface 8255 par le µp 8086.

II.2 Rappel sur les interruption

II.2.1 Définition

Les interruptions du 8086 sont des mécanismes permettant d'interrompre le programme principal pour traiter des événements externes (matériels) ou internes (logiciels/erreurs) via des routines de service (ISR).

Chaque interruption correspond à un sous-programme de service rangé dans la mémoire centrale. Lorsqu'une demande d'interruption survient, le microprocesseur a besoin de déterminer l'adresse du sous-programme associé à cette interruption. Pour cela, le périphérique qui émit l'interruption place sur le bus de données un code numérique signalant la nature de l'interruption. Le microprocesseur utilise ce code pour rechercher dans une zone en mémoire centrale, appelée tableau de vecteurs d'interruptions, l'adresse du sous-programme d'interruption à exécuter.

II.2.2 Interruptions du 8086

Le µp 8086 peut gérer jusqu'à 256 interruptions. Chaque interruption est désignée par un numéro compris entre 0 et 255 appelé type de l'interruption. On distingue trois types d'interruptions du 8086 :

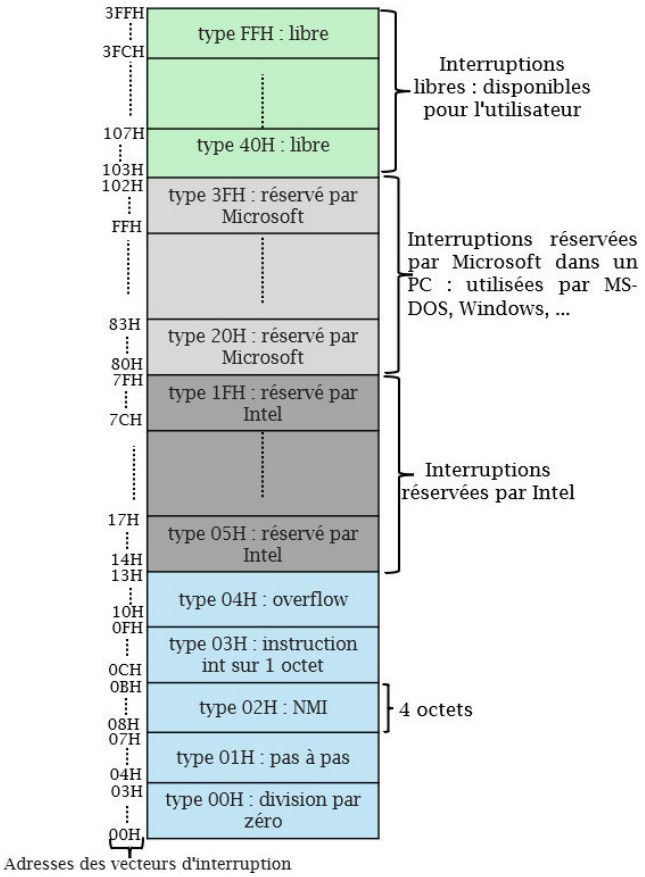


FIGURE 19 – Interruptions du 8086.

— **Interruptions matérielles** : générées par l'activation des lignes INTR (Interrupt Request) et NMI (Non-Maskable Interrupt) du microprocesseur ;

— **Interruptions logicielles** : générées par l'instruction INT n, où n est le type de l'interruption (Ex : INT 21) ;

— **Interruptions processeur** : produites par le microprocesseur en cas de division par zéro, de dépassement, ou lors de l'exécution pas à pas d'un programme.

Les interruptions du 8086 sont vectorisées. La table des vecteurs d'interruptions doit obligatoirement commencer à l'adresse physique 00000H dans la mémoire centrale. Chaque vecteur d'interruption est constitué de 4 octets représentant une adresse logique du type CS : IP.

Remarques :

- L'adresse du vecteur correspondant à une interruption est liée au numéro de l'interruption comme suit :

$$\textbf{adresse du vecteur correspondant} = 4 \times \textbf{numéro de l'interruption} \qquad (1)$$

Exemple : interruption 02H, adresse du vecteur = 4 × 02H = 08H.

- Au démarrage du système, le programme principal (carte à microprocesseur) ou le système d'exploitation (ordinateur) prend en charge la table des vecteurs d'interruptions. Elle peut être changée en cours de fonctionnement, on parle de détournement des vecteurs d'interruptions.

II.2.3 Traitement de demandes d'interruptions par le µp

Suite à une demande d'interruptions par le périphérique, le microprocesseur peut *accepter ou refuser* cette demande.

Le microprocesseur suit les étapes suivantes une fois qu'il accepte une interruption :

— Il termine l'exécution de l'instruction en cours ;
— Il sauvegarde le contenu des principaux registres sur la pile ;
— Il envoie un signal d'accusé de réception (INTA : Interrupt Acknowledge) signalant au périphérique que la demande d'interruption est acceptée.
— Il suspend l'exécution du programme en cours et va chercher à exécuter un sous-programme de service de l'interruption (ISR : Interrupt Service Routine) ;
— Une fois l'exécution de l'ISR est terminé, les registres sont restaurés à partir de la pile et le microprocesseur reprend l'exécution du programme qu'il avait suspendu.

En revanche, le processeur refuse une interruption, c'est souvent parce qu'elle est **masquée** (priorité insuffisante ou temporairement désactivée pour une tâche critique), et dans ce cas, aucun accusé de réception formel n'est envoyé pour la refuser ; le processeur continue simplement son travail. Mais, l'interruption peut rester en attente si elle n'est pas masquée de manière permanente, jusqu'à ce que le processeur soit prêt à la traiter (par exemple, si la priorité actuelle baisse).

II.3 Préparation théorique

Question sur l'interface 8255 De combien de modes fonctionnent le PPI 8255 ? Lequel des modes de 8055 est bidirectionnel ? Si $\overline{CS}$, A0 et A1 sont égaux à zéro, lequel des ports 8255 devons-nous sélectionner ? Déterminer l'état des port A0,A1, $\overline{CS}$,$\overline{RD}$ et $\overline{WR}$ pour

4. (a) pour lire les données du port A ;
   (b) pour écrire le registre de contrôle.
5. On suppose que l'interface 8255 est connectée au microprocesseur 8086 et que l'adresse de base de l'interface est 00 :
   (a) Quelles sont les adresses relatives (offsets) du Port A, Port B, Port C et du registre de contrôle ?
   (b) Déterminer le mot de commande (en binaire et en hexadécimal) pour configurer : Port A : Sortie, Port B : Entrée, Port C (haut et bas) : Sortie, Mode 0.
6. Déterminer la configuration des ports A, B, C du 8255 et le mode utilisé si le registre de contrôle contient la valeur A2H ?

   Question sur les interruptions
7. Définir une interruption.
8. Décrire brièvement les étapes effectuées automatiquement par le 8086 lorsqu'il reçoit un signal d'interruption.
9. Supposons qu'un périphérique de 8086 génère une interruption de type 10H.
   (a) À quelle adresse physique le microprocesseur va-t-il chercher le nouveau IP (Instruction Pointer) ?
   (b) À quelle adresse trouvera-t-il le nouveau CS (Code Segment) ?
10. Écrire un fragment de code assembleur 8086 qui :
    (a) Attend la saisie d'un caractère au clavier sans écho.
    (b) Affiche ce caractère suivi d'un point d'exclamation '!'.

III Manipulations

III.1 Interface 8255

III.1.1 Contrôle de clignotement d'une LED à l'aide d'un programme

Cahier des charges 1

Concevoir un système permettant de réaliser le clignotement d'une LED à l'aide du microprocesseur 8086 et l'interface parallèle 8255. Selon les contraintes suivantes :

— le clignotement se fait d'une manière automatique (par programme).

— le LED est branché sur la broche PA0

— Matériels à utiliser : µp 8086, l'interface PPI *"8255A"*, Le verrou (latch) transparent octal de type D à 8 bits *"74HC373"*, et le LED *"LED-BIRY"*.

Fiche technique 1

1. Réalisation du circuit :
   (a) Ouverture du logiciel Proteus
      — Lancer votre logiciel Proteus ;
      — Choisir un nouveau projet en lui donnant un nom et un emplacement ;
      — Cliquer sur suivant jusqu'à la page "Firmware project" : choisissez "create firmware project" ;
      — Sélectionner la famille 8086 avec le compilateur MASM32 préalablement téléchargé. Cliquer sur suivant, le "Schematic capture" et "Source coude" s'affichent automatiquement
   (b) Dans la fenêtre schématique, réaliser le système de la figure 20 en suivant les étapes suivantes :
      — Le µp 8086 avec son câblage s'affiche automatiquement avec l'ouverture du schématique, mais il reste à faire les configurations suivantes, faire un double clic sur 8086, puis :
         — Réduire la fréquence à 1 MHz
         — Dans les propriétés avancées, choisir "internal memory size" et placer le à l'adresse 0x10000
      — Ajouter l'interface PPI *"8255A"*, Le verrou (latch) transparent octal de type D à 8 bits *"74HC373"*, et le LED *"LED-BIRY"*.
      — Connecter les différents composants comme le montre la figure 20.
2. Configuration des adresses :
   — L'adresse du PORTA est 00H
   — L'adresse du PORTB est 02H
   — L'adresse du PORTC 04H
   — L'adresse du registre de commande est 06H
3. Développement du code source en assembleur.

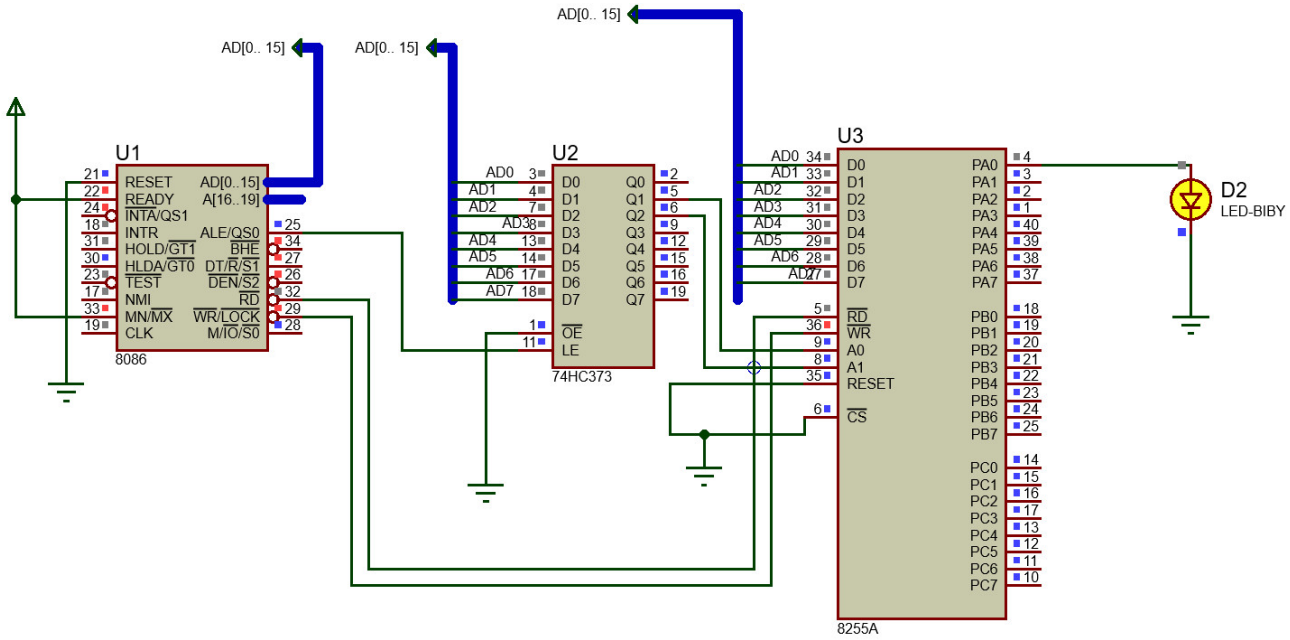


FIGURE 20 – Représentation schématique de contrôle d'une led par 8086-8255 sous Proteus.

- A l'aide d'un stylo et d'une feuille réaliser le code assembleur permettant de répondre au cahier des charges ci-dessus. La figure 21 présente l'organigramme de ce cahier de charge.
- Saisir le code source réalisé dans la fenêtre "source code" du logiciel Proteus.

4. Simulation du projet
   - Compiler le code saisi en cliquant sur l'icône "build project",
   - Allez vers la fenêtre "Schematic Capture" et lancez la simulation en cliquant sur "run simulation or F12"
   - Visualiser et conclure sur le fonctionnement du système.

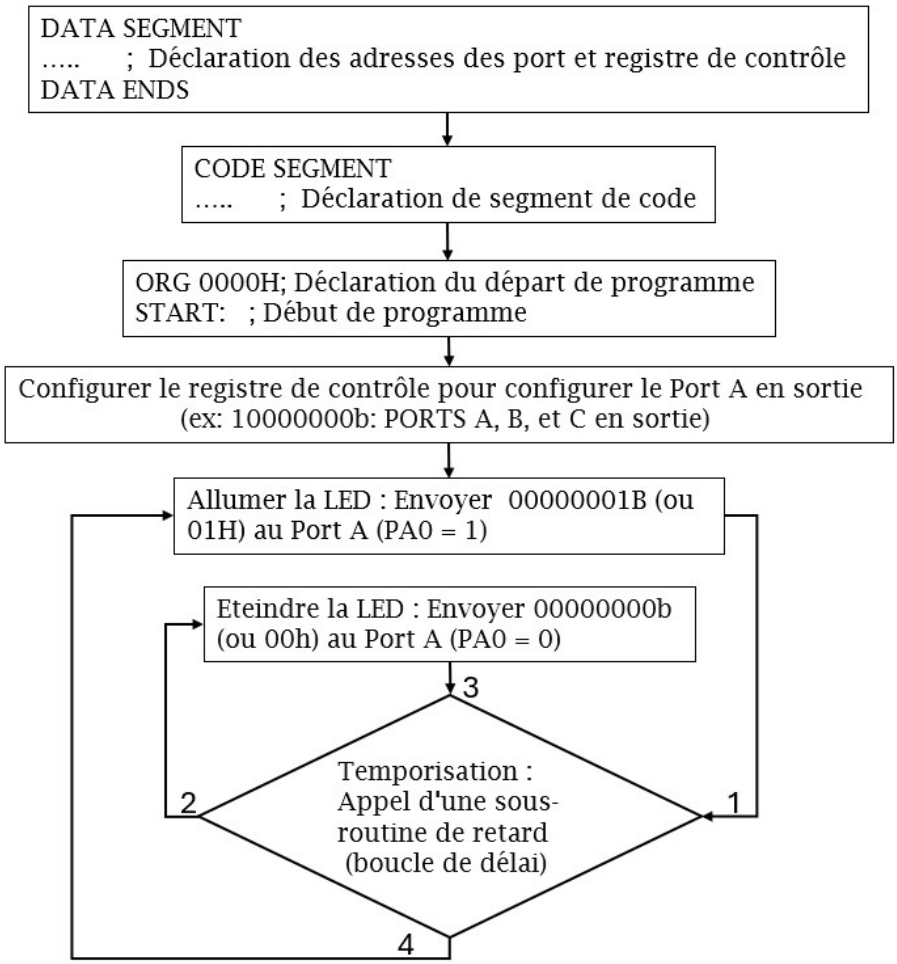


FIGURE 21 – Organigramme de clignotement d'une LED.

III.1.2 Contrôle de clignotement d'une LED à l'aide des boutons poussoir

Cahier des charges 2

Concevoir un système permet de réaliser le clignotement d'une LED à l'aide du microprocesseur 8086 et l'interface parallèle 8255. Selon les contraintes suivantes :

- La LED s'allume lorsque les commutateurs SW1 et SW3 sont fermés tandis que le commutateur SW2 est ouvert (101)
- la LED est branchée sur la broche PB0, par contre SW1, SW2, SW3 sont respectivement branchés sur les broches PA0, PA1 et PA2.
- Matériels à utiliser : µp 8086, l'interface PPI *"8255A"*, Le verrou (latch) transparent octal de type D à 8 bits *"74HC373"*, et le LED *"LED-BIRY"*, 3 boutons poussoir *"BUTTON"*, résistances, générateur d'état logique "logic state source (Latched Action)".

Fiche technique 2

1. Réalisation du circuit : suivre les mêmes étapes vues précédemment dans la fiche technique 1 pour concevoir le circuit de la figure 22.
2. Développement du code source en assembleur :
   - A l'aide d'un stylo et d'une feuille réaliser le code assembleur permettant de répondre au cahier des charges 2. La figure 23 présente l'organigramme de cahier des charges 2.

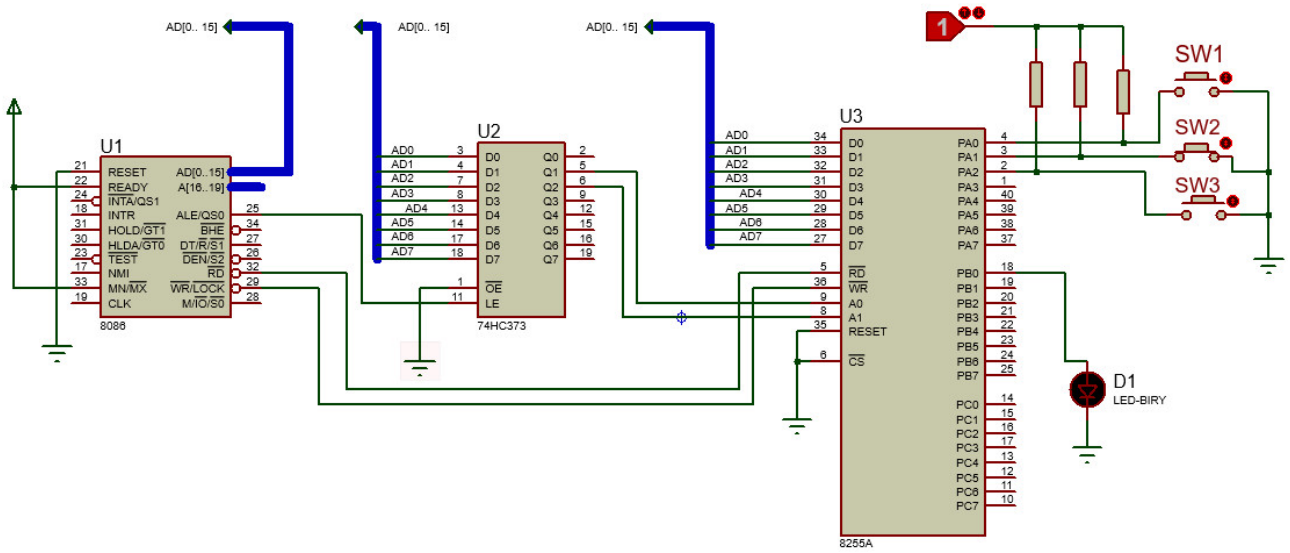


FIGURE 22 – Représentation schématique de contrôle d'une LED par trois boutons poussoir à l'aide de 8086-8255 sous Proteus.

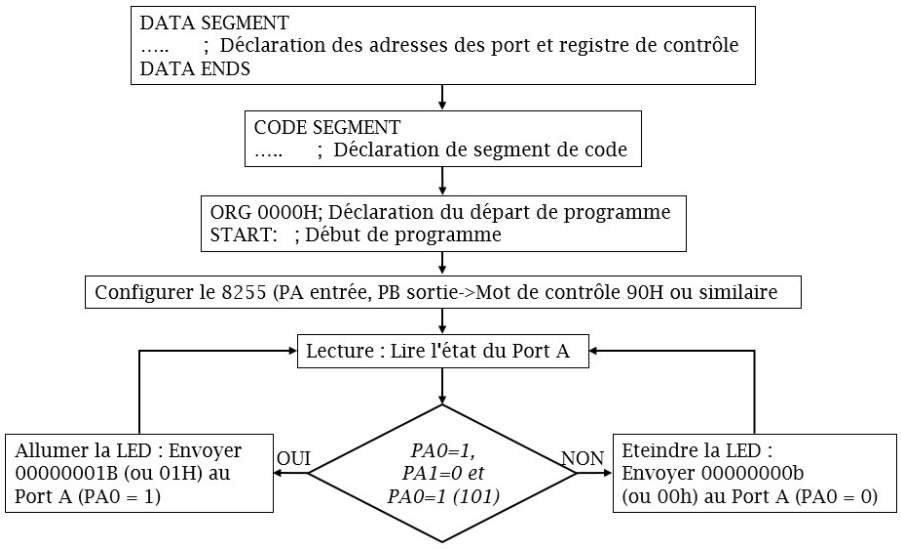


FIGURE 23 – Organigramme de clignotement d'une LED.

— Saisir le code source réalisé dans la fenêtre "source code" du logiciel Proteus.

3. Simulation du projet : Compiler le code saisi en cliquant sur l'icône "build project",
4. Allez vers la fenêtre "Schematic Capture" et lancez la simulation en cliquant sur "run simulation or F12"
5. Visualiser et conclure sur le fonctionnement du système.

III.1.3 Contrôle d'un afficheur 7-segments

Cahier des charges 3

Concevoir un système qui permet à un afficheur 7 segments d'afficher les nombres hexadécimales de 0 à F à l'aide du microprocesseur 8086 et de l'interface parallèle 8255. Selon les contraintes suivantes :

— L'affichage se déclenche d'une manière automatique dès l'exécution du programme.

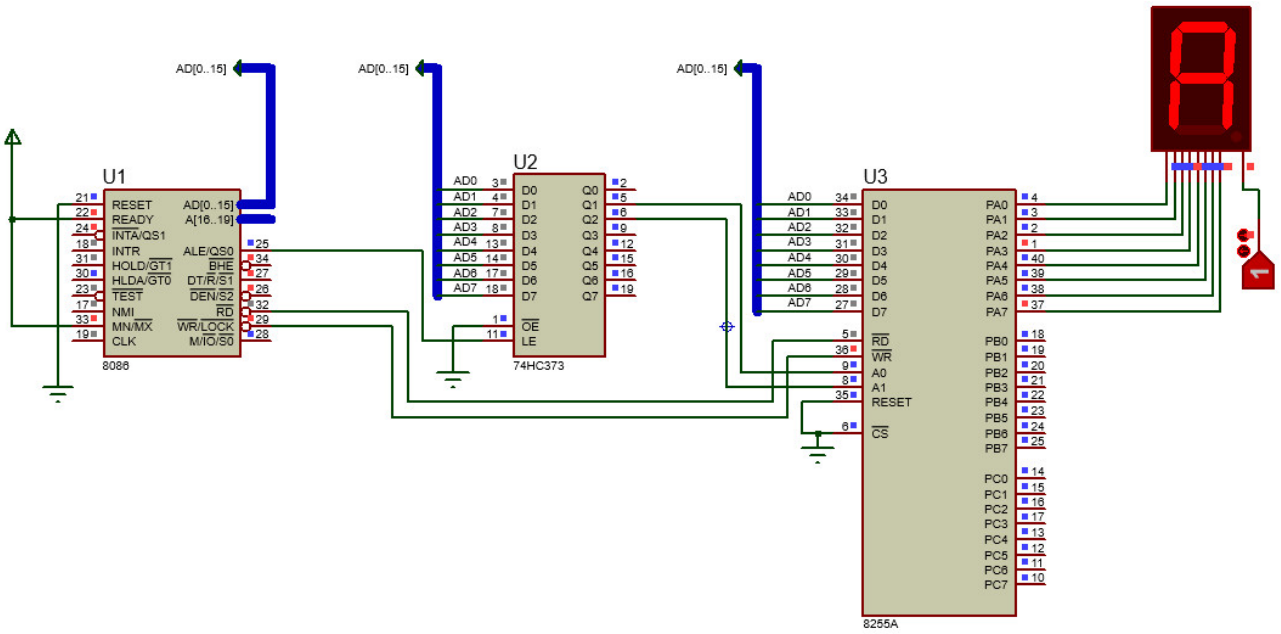


FIGURE 24 – Représentation schématique de contrôle d'un afficheur 7-segments à l'aide de 8086-8255 sous Proteus.

— L'afficheur compte de 0 à F progressivement.

— Matériels à utiliser : µp 8086, l'interface PPI *"8255A"*, Le verrou (latch) transparent octal de type D à 8 bits *"74HC373"*, générateur d'état logique "logic state source (Latched action)" et afficheur 7-segments *"7SEG-MPX1-CA"*.

Fiche technique 3

1. Réalisation du circuit : suivre les mêmes étapes vues précédemment dans la fiche technique 1 pour concevoir le circuit de la figure 24.
2. Développement du code source en assembleur :
   — A l'aide d'un stylo et d'une feuille réaliser le code assembleur permettant de répondre au cahier des charges 3. La figure 25 présente l'organigramme de cahier des charges 3.
   — Saisir le code source réalisé dans la fenêtre "source code" du logiciel Proteus.
3. Simulation du projet : Compiler le code, lancer la simulation, puis visualiser et conclure sur le fonctionnement du circuit.

Rappel : la figure 26 présente les valeurs des segments d'un afficheur 7 segments pour compter de 0 à F.

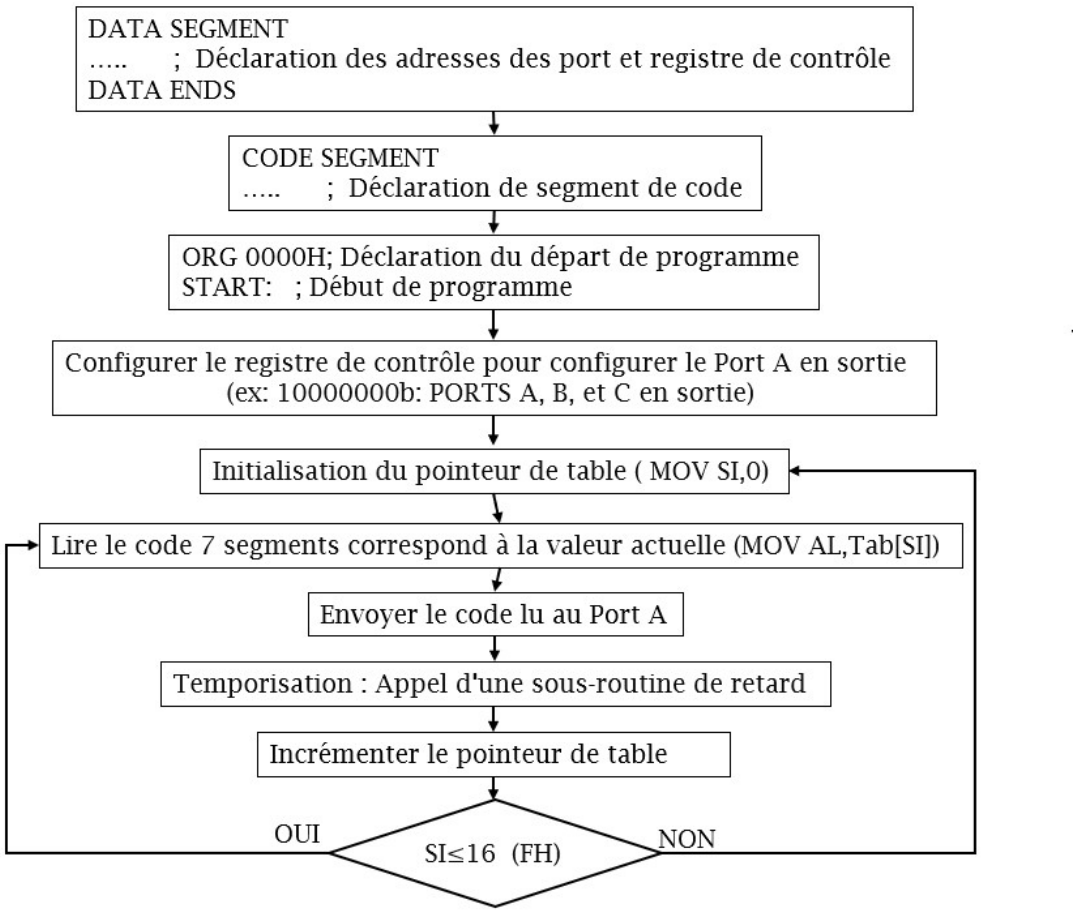
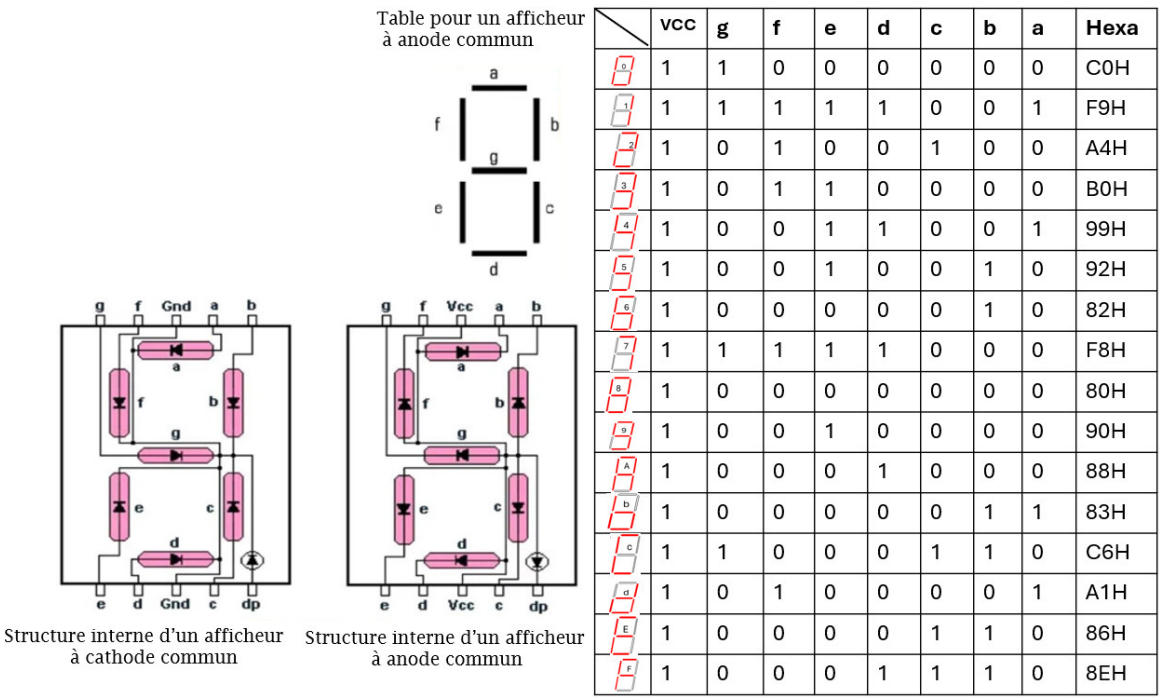


FIGURE 25 – Organigramme de clignotement d'une LED.



	vcc	g	f	e	d	c	b	a	Hexa
0	1	1	0	0	0	0	0	0	C0H
1	1	1	1	1	1	0	0	1	F9H
2	1	0	1	0	0	1	0	0	A4H
3	1	0	1	1	0	0	0	0	B0H
4	1	0	0	1	1	0	0	1	99H
5	1	0	0	1	0	0	1	0	92H
6	1	0	0	0	0	0	1	0	82H
7	1	1	1	1	1	0	0	0	F8H
8	1	0	0	0	0	0	0	0	80H
9	1	0	0	1	0	0	0	0	90H
A	1	0	0	0	1	0	0	0	88H
b	1	0	0	0	0	0	1	1	83H
c	1	1	0	0	0	1	1	0	C6H
d	1	0	1	0	0	0	0	1	A1H
E	1	0	0	0	0	1	1	0	86H
F	1	0	0	0	1	1	1	0	8EH

FIGURE 26 – Table d'un afficheur 7-segments à anode commun.

III.2 Interruptions

Dans la suite de ce TP, on s'intéresse à la manipulation de quelques interruptions du µp 8086. Pour cela, nous allons utiliser le logiciel EMU8086.

▷ Interruption : INT 21h (Fonction AH=01h)

1. Lancer l'environnement Emu8086 et appuyer sur l'onglet New. Puis choisissez .COM template.
2. Taper et sauvegarder le programme suivant :

```
org 100h
mov ah,1
int 21h
ret
```

3. Émulez et exécutez-le.
   (a) Que remarquez-vous concernant le registre AL ? Que représente ce contenu ?
   (b) Déduire le rôle de cette interruption.

▷ Interruption : INT 21h (Fonction AH=02h)

1. Taper et sauvegarder le programme suivant :

```
org 100h
mov dl,'k' mov ah,2
int 21h
ret
```

2. Émulez et exécutez-le.
   (a) Que remarquez-vous concernant le registre AL ? Que représente ce contenu ?
   (b) Déduire le rôle de cette interruption.

▷ Interruption : INT 21h (Fonction AH=05h)

1. Taper et sauvegarder le programme suivant :

```
org 100h
mov dl,'k' mov ah,5
int 21h
ret
```

2. Émulez et exécutez-le.
   (a) Que remarquez-vous concernant le registre AL ? Que représente ce contenu ?
   (b) Déduire le rôle de cette interruption.

▷ Interruption : INT 21h (Fonction AH=09h)

1. Taper et sauvegarder le programme suivant :

```
org 100h
msg db 'Hello, it is me !$'
mov dx, offset msg
mov ah, 9
int 21h
ret
```

2. Émulez et exécutez-le.
3. Déduire le rôle de cette interruption.

▷ Interruption : INT 21h (Fonction AH=2Ah)

1. Taper et sauvegarder le programme suivant :

```
org 100h
mov ah, 2Ah
int 21h
ret
```

2. Émulez et exécutez-le.
3. Vérifier que cette interruption permet de générer la date système avec retour : CX = année (1980-2099). DH = mois. DL = jour. AL = jour de la semaine (00h = dimanche).

▷ Interruption : INT 21h (Fonction AH=2Ch)

1. Taper et sauvegarder le programme suivant :

```
org 100h
mov ah, 2Ch
int 21h
ret
```

2. Émulez et exécutez-le.
3. Vérifier que cette interruption permet de générer l'heure système avec retour : CH = heure, CL = minute, DH = seconde et DL = 1/100 de seconde.

Partie II

Systèmes à microcontrôleur

TP 3 : Contrôle des LEDs et Détection d'objets à l'aide de capteur Ultrason HC-SR04

I Objectifs de TP

L'objectif principal de ce TP est de concevoir un système intelligent avec Arduino Uno et capteurs ultrason HC-SR04 capable de détecter des obstacles et de mesurer leur distance. A l'aide de ce capteur, le système peut détecter la présence d'un obstacle et déclencher une action via des LED.

A l'issue de ce TP, l'étudiant doit comprendre :

— Le fonctionnement des capteurs ultrason (HC-SR04) ;

— Implémentation avec Arduino : Programmer la carte pour lire les données des capteurs et contrôler des actionneurs ;

— Logique d'évitement/Alerte : Créer un algorithme qui déclenche une LED si un obstacle est trop proche.

II Rappel

II.1 Rappel sur la carte Arduino Uno

II.1.1 Définition

ARDUINO UNO est une carte électronique dont le coeur est un microcontrôleur ATMEL de référence ATMega 328. Ce dernier est un microcontrôleur 8 bits de la famille AVR, dont la programmation peut être réalisée en langage C/C++.

L'avantage principal des cartes Arduino est leur facilité de mise en œuvre. Ils disposent d'un environnement de développement (IDE) qui appuie sur des outils open-source. En outre, le transfert du programme compilé dans la mémoire du microcontrôleur se fait très simplement à l'aide d'un port USB. Enfin, plusieurs bibliothèques de fonctions sont également disponibles pour l'exploitation des E/S courantes : génération de signaux PWM, gestion des convertisseurs ADC, E/S TOR, exploitation de servomoteurs, d'afficheurs LCD, exploitation de bus TWI/I2C...

II.1.2 Présentation de la carte Arduino Uno

La figure 27 présente la carte Arduino Uno.

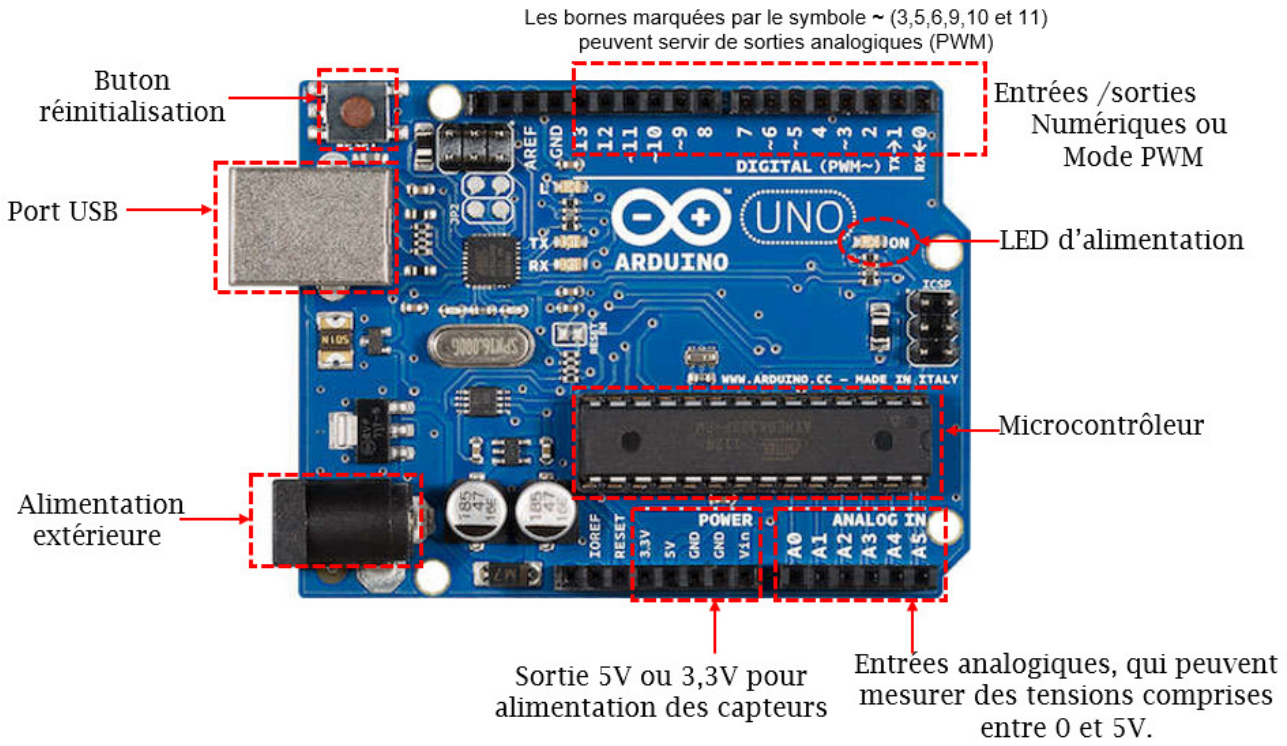


FIGURE 27 – Présentation de la carte Arduino Uno.

Les principales caractéristiques de cette carte sont résumées dans le tableau 4.

Caractéristiques	Valeur/Types
Microcontrôleur	ATmega328
- Fréquence d'horloge	16 MHz
- Tension de sortie	5 V
- Tension d'entrée (recommandée)	7-12 V
- Tension d'entrée (limites)	6-20 V
- Ports numériques	14 Entrées/sorties
- Ports analogiques	6 entrées
- Courant max par broche d'E/S	40 mA
- Courant max par broche 3.3 V	50 mA
- Mémoires	32 Ko Flash, 2 Ko SRAM, 1 Ko EEPROM
- Chargeur d'amorçage	0.5 Ko (en mémoire Flash)
- Interface	USB
- Dimensions	6.86 cm × 5.3 cm
- Prix	250 MAD

TABLE 4 – Caractéristiques de la carte Arduino Uno.

II.2 Programmation de la carte Arduino Uno

II.2.1 Environnement de développement (IDE)

La programmation de la carte Arduino se fait via l'environnement de développement (IDE, Integrated Development Environment en anglais) constituant d'un éditeur de source, des bou-

tons pour les opérations de compilation et de chargement des programmes (sketches) dans la mémoire du microcontrôleur (voir la figure 28). Le fichier de l'installation de l'IDE ainsi que les drivers de la carte Arduino sont téléchargeables librement sur : https ://docs.arduino.cc/software/

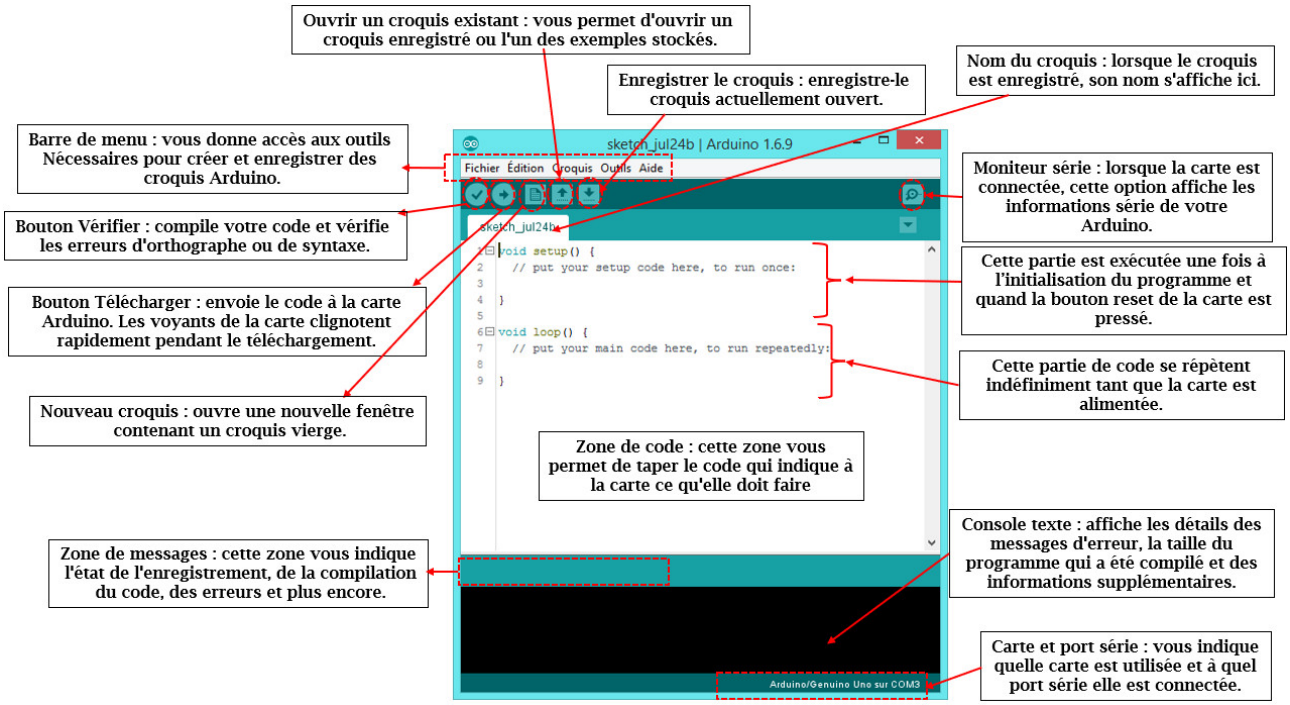


FIGURE 28 – Description de l'IDE de la carte Arduino.

II.2.2 Programmation et configuration de la carte Arduino

La programmation et la configuration de la carte Arduino se font en trois étapes :

Etape 1 : **Préparation et configuration**

- Installer l'IDE Arduino : Téléchargez et installez le logiciel Arduino IDE depuis le site officiel https ://docs.arduino.cc/software/
- Connecter la carte : Branchez l'Arduino Uno avec l'ordinateur via un câble USB (voir la figure 29).
- Configurer l'IDE : Dans le menu Outils > Type de carte, choisissez "Arduino Uno". Ensuite, dans Outils > Port, sélectionnez le port série correspondant à la carte utilisée (voir la figure 29).

Etbpe 2 : **Écriture du code (Sketch)** : Un code Arduino (sketch) contient généralement :

- Des déclarations de variables (optionnelles).
- La fonction void setup() : s'exécute une seule fois au démarrage pour initialiser les broches et les composants.
- La fonction void loop() : s'exécute indéfiniment tant que la carte Arduino est fonctionnée.

Etcpe 3 : Compilation et téléversement du code

- Vérifier/Compiler : Cliquez sur l'icône de coche $\surd$ pour vérifier la syntaxe de code et le compiler.

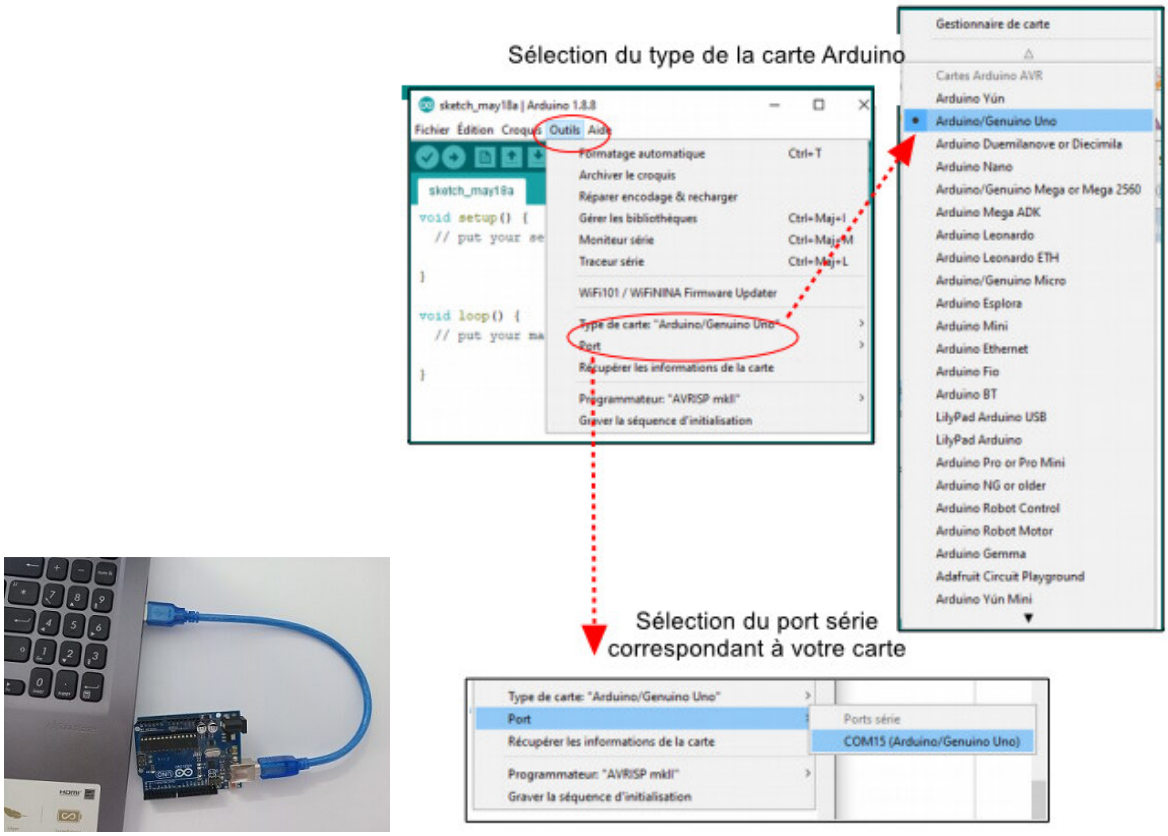


FIGURE 29 – Configuration de l'IDE de l'Arduino.

— Téléverser : Cliquez sur l'icône de flèche droite ⟶ pour envoyer le programme compilé (sketch) vers la carte Arduino.

II.2.3 Langage Arduino

Le langage Arduino est très proche des langages C et C++ auxquels s'ajoutent les fonctions des nombreuses bibliothèques (libraries) Arduino. On distingue entre les fonctions, les variables et les opérateurs :

▷ **Fonctions** : sont utilisées pour structurer le programme. Le tableau 5 résume les principales fonctions de langages Arduino.

▷ **Variables** (types int, float, boolean, char) : sont utilisées dans un programme pour stocker des données. Le tableau (6) regroupe les différents types de variables et types de données.

▷ **Opérateurs** : sont utilisés pour réaliser les calculs et les conditions (if, for, while). Le tableau (7) présente les principaux opérateurs de langage Arduino.

Fonction	Description
void setup()	S'exécute une seule fois au démarrage pour initialiser les broches, la communication série, etc..
void loop()	S'exécute en continu après setup(), c'est le cœur du programme
digitalWrite(pin, valeur)	Définit une broche numérique sur HIGH (5V) ou LOW (0V)
pinMode(pin, mode)	Définit une broche comme INPUT ou OUTPUT
analogRead(pin)	Lit une valeur analogique (0-1023) sur une broche analogique (A0-A5).
digitalRead(pin)	Lit l'état (HIGH/LOW) d'une broche numérique
analogWrite(pin, value)	Envoie une valeur PWM (0-255) à une broche PWM (tilde).
Serial.begin(baudrate) / Serial.println()	Pour la communication série avec un ordinateur.
delay(ms)	Met le programme en pause pour un nombre de millisecondes spécifié.

TABLE 5 – Fonctions principales du langage Arduino.

Variable et types de donnée	Description
int	Entiers (ex : 10, -50)
float	Nombres décimaux (ex : 3.14)
boolean	true ou false (vrai/faux)
char	Caractères (ex : 'A')
byte	Entiers non signés de 0 à 255
unsigned long	Pour les grands nombres, souvent utilisé pour le temps (millis())

TABLE 6 – Variables et types de données dans le langage Arduino.

Opérateur	Description
Opérateurs arithmétiques	+, -, *, /, % (modulo)
Opérateurs de comparaison	== (égal), != (différent), < (inférieur), > (supérieur), <= , >=
Opérateurs logiques	&& (ET), || (OU), ! (NON)
Conditions	if (condition) ... else ...
Boucles	for (init ; condition ; incrément) ... , while (condition) ...
Constantes	HIGH, LOW, INPUT, OUTPUT, true, false

TABLE 7 – Opérations et structures de contrôle dans le langage Arduino.

Remarques :

— Pour se familiariser avec le langage Arduino plusieurs exemples pré-enregistrés peuvent être chargés et testés : pour ouvrir un exemple sur l'IDE allez vers : Fichier > Exemples >01.Basics > ...

— Langage Arduino se différencie entre les majuscules et les minuscules

— En langage Arduino, la plupart des lignes se termine par un point virgule ;

— Les blocs, boucles, fonctions sont encadrées par des accolades { .}, apprendre à bien les repérer !

— Toutes les variables doivent être déclarées (nom et type).

— L'indentation (décalage) n'est pas obligatoire mais conseillée pour bien se repérer dans le programme.

— Les commentaires sont définis par // en début de ligne. Les blocs de commentaires par /* au début et */ à la fin

Exemple : Clignoter la LED intégrée (broche 13) sur Arduino

```
void setup() {
  pinMode(13, OUTPUT); // Initialise la broche 13 comme une sortie (pour la LED)
}

void loop() {
  digitalWrite(13, HIGH); // Allume la LED (état HAUT)
  delay(1000);            // Attend 1000 millisecondes (1 seconde)
  digitalWrite(13, LOW);  // Éteint la LED (état BAS)
  delay(1000);            // Attend 1000 millisecondes (1 seconde)
}
```

II.3 Rappel sur le Capteur ultrason HC-SR04

II.3.1 Définition

HC-SR04 est un capteur à ultrasons qui est principalement utilisé pour la mesure de distance. En émettant des ondes ultrasonores et en mesurant le temps pris par ces ondes pour rebondir après avoir frappé un objet, il peut déterminer avec précision la distance à laquelle un objet se trouve. De par son coût abordable et sa facilité d'intégration avec des plateformes, telles qu'Arduino, HC-SR04 est devenu un choix prisé parmi les amateurs de bricolage et les professionnels.

Le tableau 8 décrit les caractéristiques techniques du capteur ultrason HC-SR04.

II.3.2 Principe de fonctionnement

Pour déclencher une mesure, il faut présenter une impulsion "high" (5 V) d'au moins 10 µs sur l'entrée "Trig". Le capteur émet alors une série de 8 impulsions ultrasoniques à 40 kHz, puis il attend le signal réfléchi. Lorsque celui-ci est détecté, il envoie un signal "high" sur la sortie "Echo", dont la durée est proportionnelle à la distance mesurée (31).

Le temps mis pour parcourir un aller-retour permet de déterminer la distance de l'objet par rapport à la source du premier écho. Plus l'objet sera loin, plus il faudra longtemps pour que

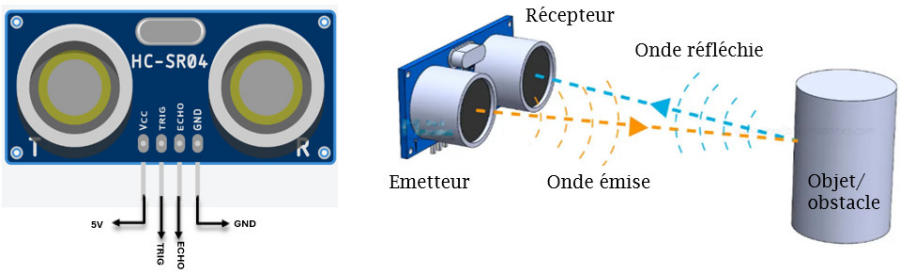


FIGURE 30 – Architecture externe et principe de fonctionnement du capteur ultrason.

Caractéristique	Valeur
Courant de fonctionnement	15 mA
Tension de fonctionnement	5 V DC
Fréquence de fonctionnement	40 Hz
Portée maximale	4 m
Portée minimale	2 cm
Angle de mesure	15 degrés
Signal d'entrée de déclenchement impulsion	TTL de 10 µs
Signal de sortie d'écho signal de niveau	TTL d'entrée proportionnel à la distance
Dimensions	45 × 20 × 15 mm

TABLE 8 – Caractéristiques techniques du capteur à ultrasons

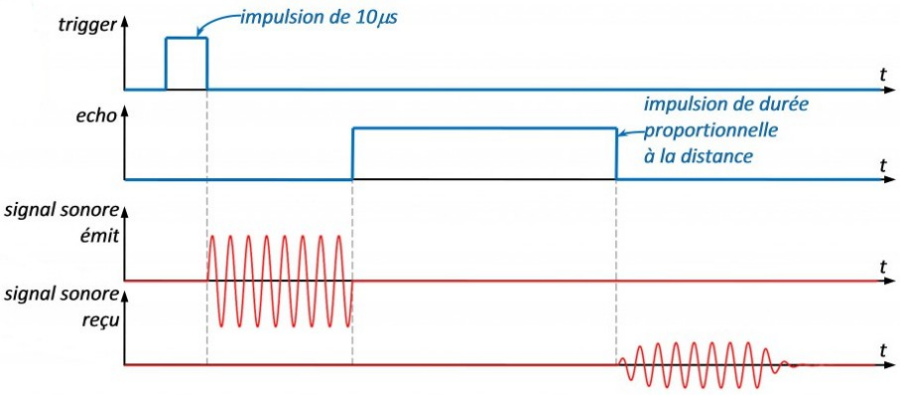


FIGURE 31 – Principe de fonctionnement du capteur ultrason HC-SR04.

le signal revienne. Pour détecter le niveau (la distance d), on utilise l'équation suivante.

$$v = \frac{2.d}{t} \Longrightarrow d = \frac{v.t}{2} \tag{2}$$

avec t étant la durée entre le début de l'émission et le début de la réception et v étant la vitesse du son dans l'aire qui est de 343 m/s.

II.4 Signal PWM

Contexte : Limite des sorties analogiques sur Arduino

L'Arduino possède des broches analogiques numérotées de A0 à A5. Ces broches peuvent uniquement être utilisées comme entrées analogiques, mais ne peuvent pas générer de signaux analogiques. Cela s'explique par la nature numérique du microcontrôleur, qui ne sait produire que des signaux HAUT (5 V) ou BAS (0 V). Alors comment peut-on alors générer un signal analogique avec une carte numérique ?

Solution : Signal PWM

Il existe une méthode qui permet de générer un signal similaire à un signal analogique en utilisant uniquement des signaux numériques : c'est la modulation de largeur d'impulsion, connue sous le nom de PWM (Pulse Width Modulation).

Le PWM fonctionne en envoyant une série d'impulsions (ON/OFF) très rapides. Le rapport entre la durée pendant laquelle le signal reste à l'état haut (ON), T_{ON}, et sa période totale, T, est appelé le rapport cyclique (duty cycle), souvent noté α, exprimé en pourcentage :

$$\alpha = \frac{T_{ON}}{T}100\% = \frac{T_{ON}}{T_{ON} + T_{OFF}}100\% \tag{3}$$

avec T_{OFF} c'est la durée pendant laquelle le signal reste à l'état bas , OFF.

Par exemple :

- 0% $\Longrightarrow$ le signal est toujours à 0 (OFF)
- 50% $\Longrightarrow$ le signal est ON la moitié du temps, OFF l'autre moitié
- 100% $\Longrightarrow$ le signal est toujours ON

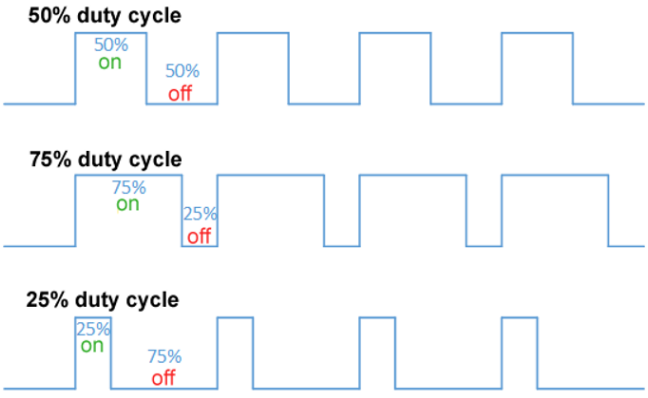


FIGURE 32 – PWM et Rapport cyclique.

En effet, un rapport cyclique de 50% (le signal est haut pendant la moitié du temps) donne une tension moyenne d'environ 2.5 V. Grâce à cette technique, on peut contrôler progressivement des dispositifs comme :

— L'intensité lumineuse d'une LED ;

— La vitesse d'un moteur à courant continu ;

— Le niveau d'un signal sonore.

II.5 Préparation théorique

Question sur l' Arduino Quelles sont les deux fonctions obligatoires dans tout programme Arduino ? Quelle instruction permet de définir si une broche (pin) est une entrée ou une sortie ? À quoi sert la fonction setup() ? S'exécute-t-elle en boucle ou une seule fois ? Quelle est la différence entre les broches numériques et analogiques d'une carte Arduino ?Comment allumer une LED sur la broche 13 ?Quelle broche permet de lire la valeur d'un potentiomètre ? À quoi servent les broches marquées d'un tilde ~ ?Sur quel port branche-t-on un bouton poussoir ?

Question sur le capteur ultrason HC-SR04 Quel est le principe physique utilisé par le capteur HC-SR04 ? À quoi servent les broches "Trigger" et "Echo" ? Quelle est la fréquence des ondes ultrasoniques émises ? Quelle est la formule pour calculer la distance avec ce capteur ? Pourquoi divise-t-on le temps par 2 dans cette formule ?

III Manipulation

III.1 Contrôle le clignotement d'une LED

Cahier des charges 1

Concevoir un système permettant de réaliser le clignotement d'une LED à l'aide de l'Arduino Uno. Selon les contraintes suivantes :

1. Le clignotement se fait d'une manière automatique (par programme), 500 ms ON, 500 ms OFF.

— La LED est branchée sur la broche 3.

— Matériels à utiliser : carte Arduino, LED, un PC, câble USB, résistance de 1 kΩ.

Fiche technique 1

1. Réaliser le circuit de la figure 33.
2. Développement du code source :
   — Lancer l'environnement IDE dans votre PC ;
   — Choisir un nouveau projet en lui donnant un nom et un emplacement ;
   — Écrire un programme Arduino permettant de faire clignoter la LED. *Fonctions prédéfinies à utiliser sont définies dans la section II.2.3.*
3. Simulation du projet
   — Compiler le code saisi en cliquant sur l'icône "Vérifier".
   — Envoyer le code à la carte Arduino en cliquant sur l'icône "Télécharger".
   — Visualiser et conclure sur le fonctionnement du système.
   — Parmi les LEDs de signalisation présentes sur la carte Arduino, identifier celle qui suit le même clignotement que notre LED.

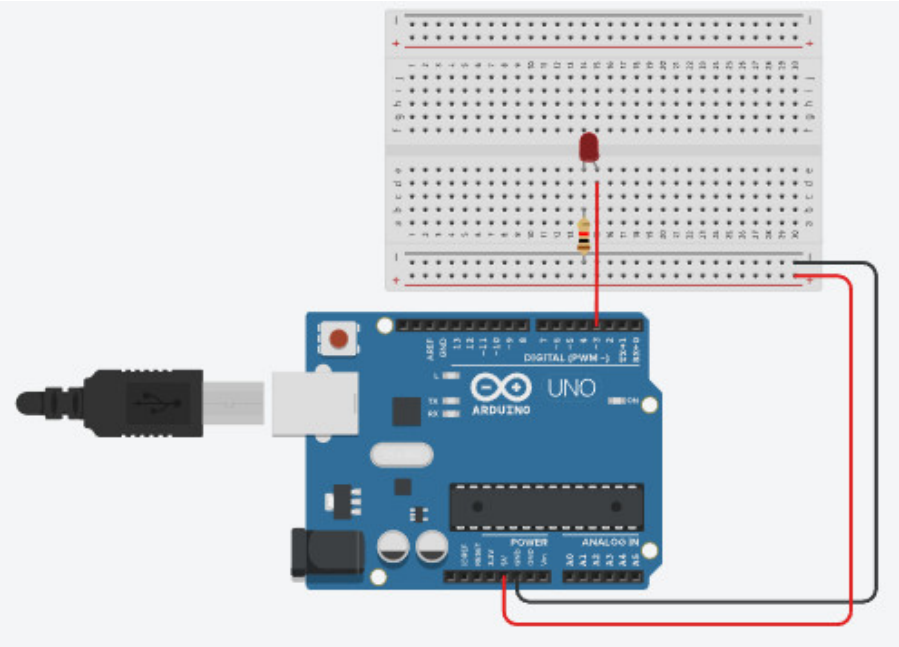


FIGURE 33 – Contrôle de clignotement d'une LED par Arduino Uno

III.1.1 Contrôle de l'intensité lumineuse d'une LED

Cahier des charges 2

Concevoir un système permettant de contrôler l'intensité lumineuse d'une LED à l'aide de l'Arduino Uno. Selon les contraintes suivantes :

- Faire varier l'intensité d'une LED, suivant les valeurs d'intensité suivantes : 0%, 25%, 50%, 75% et 100% ;
- Faire varier l'intensité d'une LED de manière ascendante et ensuite descendante, en utilisant une boucle for ;
- La LED est branchée sur la broche 3 ;
- Matériels à utiliser : carte Arduino, LED, un PC, câble USB, résistance de 1 kΩ.

Fiche technique 2

1. Réaliser le circuit de la figure 33.
2. Développement du code source :
   - Lancer l'environnement IDE dans votre PC ;
   - Choisir un nouveau projet en lui donnant un nom et un emplacement ;
   - Écrire un programme Arduino permettant de varier l'intensité lumineuse d'une LED, suivant les valeurs d'intensité suivantes : 0%, 25%, 50%, 75% et 100%. *Fonctions prédéfinies à utiliser sont définies dans la section II.2.3.*
3. Simulation du projet
   - Compiler le code saisi en cliquant sur l'icône "Vérifier".
   - Envoyer le code à la carte Arduino en cliquant sur l'icône "Télécharger".
   - Visualiser et conclure sur le fonctionnement du système.

— Parmi les LEDs de signalisation présentes sur la carte Arduino, identifier celle qui suit le même clignotement que notre LED.

4. Écrire puis simuler un nouveau programme permettant de varier l'intensité d'une LED de manière ascendante et ensuite descendante, en utilisant une boucle for ;

III.1.2 Détection d'objets à l'aide de Capteur Ultrason HC-SR04

Cahier des charges 3

Concevoir un système permettant de détecter la présence d'un objet dans la plage de mesure du capteur et déclencher une LED si un obstacle se trouve à une distance de 50 cm. Selon les contraintes suivantes :

— Les broches ECHO et TRIG de capteur sont respectivement branchées sur les broches 8 et 9 de l'Arduino Uno ;

— La LED1 et LED2 sont branchées respectivement sur les broches 2 et 3 de l'Arduino ;

— Matériels à utiliser : carte Arduino, LED, un PC, câble USB, capteur ultrason HC-SR04, résistances de 1 kΩ.

Fiche technique 3

1. Réaliser le circuit de la figure 34 **sans les LEDs**.

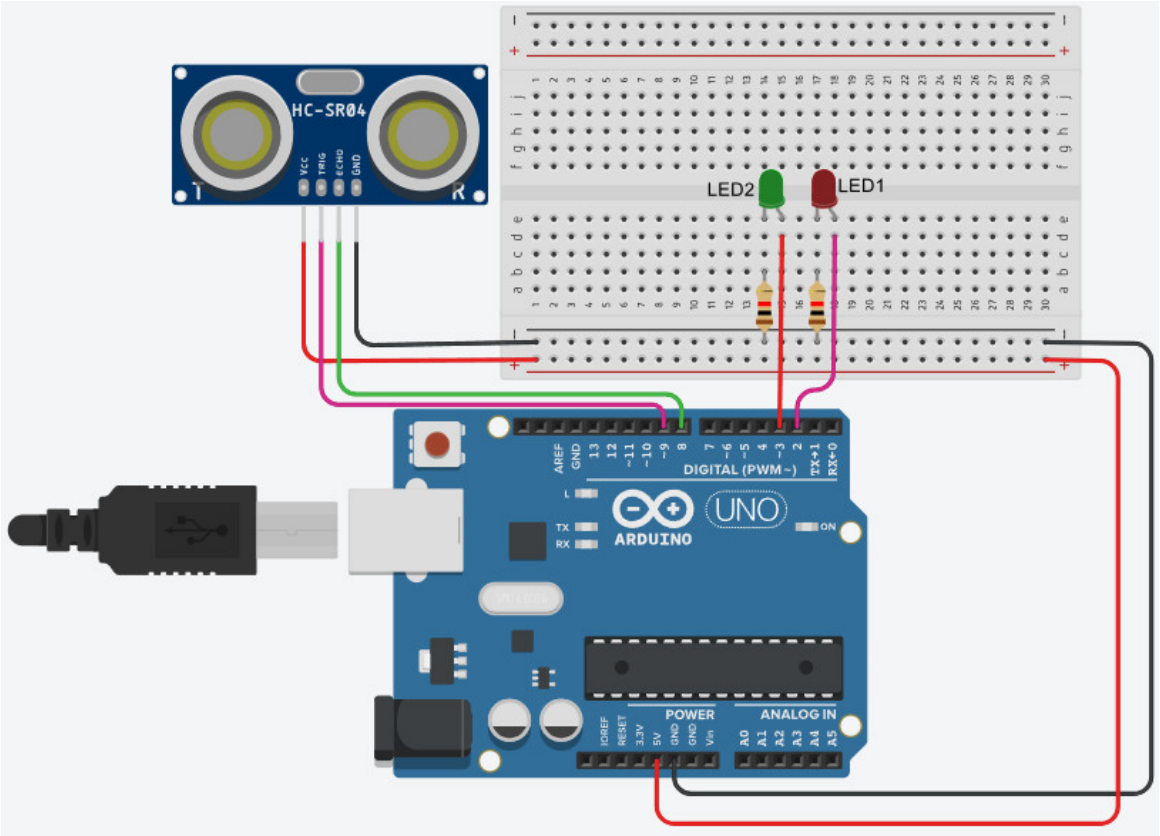


FIGURE 34 – Mesure de distance par capteur ultrason HC-SR04 avec Arduino Uno.

2. Développement du code source :

— Lancer l'environnement IDE dans votre PC ;

— Choisir un nouveau projet en lui donnant un nom et un emplacement ;

— Écrire un programme qui permet de détecter un objet : mesurer la distance entre le capteur et l'objet, puis l'afficher sur le moniteur série sous la forme suivante : « La distance est ..... cm ». En plus des fonctions définies dans la section II.2.3, vous aurez besoin des fonctions suivantes :

- pulseIn() : Lire une impulsion (soit HIGH ou LOW) sur une broche. Par exemple, si la valeur est HIGH, pulseIn() attend que la broche passe de LOW à HIGH, démarre le chronométrage, puis attend que la broche passe à LOW et arrête le chronométrage. Renvoie la longueur (T) de l'impulsion en microsecondes. *Syntax : unsigned long T = pulseIn(broche, valeur) ;* avec Broche : le numéro de la broche Arduino sur laquelle vous connectez la broche Echo et Valeur : type d'impulsion à lire (soit HIGH ou LOW).
- Serial.begin(9600), initialiser la liaison Série à 9600 bauds ;
- Serial.print("msg") ou Serial.println() ; afficher le texte msg sur le moniteur série.
- map(valeur, fromLow, fromHigh, toLow, toHigh) ; convertir (ou "mapper") une valeur numérique d'une plage d'entrée donnée vers une nouvelle plage de sortie proportionnelle. Exemple : convertir la lecture d'un potentiomètre (0-1023) en une valeur PWM pour une LED (0-255) ; int valeurPWM = map(analogRead(A0), 0, 1023, 0, 255) ;

Algorithme à suivre :

⋆ Configurer Trig en sortie et Echo en entrée.

⋆ Envoyer une impulsion brève (LOW 2 µs, puis HIGH 10 µs, puis LOW 2 µs) sur le Trig.

⋆ Mesurer la durée t (en µs) pendant laquelle Echo est HIGH

⋆ Calculer la distance d en cm :

$$d(cm) = \frac{t * 0.0343}{2} \tag{4}$$

⋆ Afficher la distance dans le moniteur série

3. Simulation du projet
   — Compiler le code saisi en cliquant sur l'icône "Vérifier".
   — Envoyer le code à la carte Arduino en cliquant sur l'icône "Télécharger".
   — Déplacer un objet devant le capteur ultrasonique et vérifier que la distance affichée par le moniteur série augmente lorsque l'objet s'éloigne et diminue quand il se rapproche. Conclure
4. **Ajouter au circuit précédent la LED 1** (N.B. *Avant d'ajouter la LED1, veuillez débrancher la carte Arduino)*
   — Modifier le programme précédent en ajoutant des instructions qui permettent de déclencher la LED1 si d est inférieur à 100 cm ;
   — Simuler ce nouveau programme. Conclure.
5. **Ajouter au circuit précédent la LED 2** (N.B. *Avant d'ajouter la LED2, veuillez débrancher la carte Arduino)*
   — Modifier le programme précédent en ajoutant des instructions permettant de déclencher la LED1 si $d < 100$ cm et de varier l'intensité de la LED2 de manière ascendante si l'objet se rapproche et descendante si l'objet s'éloigne.
   — Simuler ce nouveau programme. Conclure.